

Harry Myers

CS m146 HW#3

1. $J(\mu, z) = \sum_{n=1}^N \|x_n - \mu_{z_n}\|^2$ $x_n \in \mathbb{R}^d$, $z_n \in \{1, \dots, k\}$ cluster, $\mu_c \in \mathbb{R}^d$ is a centroid K means

Lloyd's 1. Assignment - Centroids fixed 2. update each z_n to minimize J . (change points, clusters)

2. Update - Assignments Fixed, update each centroid μ_c to minimize J .

a. Show that the value that minimizes J is the mean of the points assigned to C .

$$\mu_c^* = \frac{1}{|C_c|} \sum_{n: z_n=c} x_n \quad ; \quad C_c = \{n: z_n=c\} \quad , \quad J = \sum \|x_n - \mu_c\|^2 + \text{terms not involving } \mu_c$$

$$\nabla \mu_c = -2 \sum_{n: z_n=c} (x_n - \mu_c) = 0 \quad \sum_{n: z_n=c} x_n = \sum_{n: z_n=c} \mu_c \quad \text{all labels } n \text{ s.t. point } n \text{ is assigned to cluster } C$$

$$\sum_{n: z_n=c} x_n = |C_c| \mu_c \quad \text{so} \quad \mu_c^* = \frac{1}{|C_c|} \sum_{n: z_n=c} x_n$$

b. Fixed centroids $\rightarrow$ best cluster for each x_n $J = \sum \|x_n - \mu_{z_n}\|^2 = \|x_1 - \mu_{z_1}\|^2 + \|x_2 - \mu_{z_2}\|^2 + \dots$ only depends on a constant

$$\text{so for point } x_n \text{ just pick the center that's closest: } z_n^* = \underset{c \in \{1, \dots, k\}}{\text{argmin}} \|x_n - \mu_c\|^2$$

c. I. monotonically decreases II. Converges. J decreases after assignment step because if any point gets reassigned to a closer centroid J strictly decreases. For update step assignments are fixed and move each centroid to mean of its cluster so optimal centroid so J must decrease

so $J^{t+1} \leq J^t$. There are only a finite # of ways to assign n points to k clusters so we can't

never revisit some one because J would have to be equal but J is already lower. so Alg must eventually converge.

d. why J converges to local min? Lloyd 1985 is a non-convex func so it descends on one of many minimizers none of which are guaranteed to be the global min. This is why randomizing the starting centroids position can lead to different results often better ones than on the first attempt.

2. PCA; Variance maximization & singular value decomposition

$X \in \mathbb{R}^{N \times d}$ X_c is centered. (each col has mean of 0) $Cov = C = \frac{1}{N-1} X_c^T X_c \in \mathbb{R}^{d \times d}$

a. First PC max variance of projection: $V \in \mathbb{R}^d$, Proj taken $z_n = V^T(x_n - \bar{x}) = \text{vector}^T(\text{centered data})$

I. Show var $z_1, \dots, z_n = V^T C V$ $z_n = \text{scalar}$ How far x_n projects onto V , $\text{var}(z) = \frac{1}{N-1} \sum (z_n - \bar{z})^2$

$$\bar{z} = \frac{1}{N} \sum z_n = \frac{1}{N} \sum V^T(x_n - \bar{x}) = V^T \frac{1}{N} \sum (x_n - \bar{x}) = 0 \quad \text{so } \bar{z} = 0 \quad \text{var}(z) = \frac{1}{N-1} \sum z_n^2$$

$$= V^T \left(\frac{1}{N-1} \sum (x_n - \bar{x})^T (x_n - \bar{x}) \right) V = V^T C V$$

II. max $V^T C V$, $V^T V = 1$ $L(V, \lambda) = V^T C V - \lambda(V^T V - 1)$ $P.L. = 2CV - 2\lambda V = 0$ $(CV = \lambda V)$

III. pick eigenvector with largest eigenvalue? C is $d \times d$ which ones? $V^T C V = V^T \lambda V = \lambda$ so eigenvector with largest eigenvalue max variance

b. SVD: $X_c = U \Sigma V^T$ $U \in \mathbb{R}^{N \times r}$, $\Sigma \in \mathbb{R}^{r \times r}$ diagonal, $V \in \mathbb{R}^{d \times r}$ $r = \min(N, d)$ $U^T U = V^T V = I$

I. $X_c^T = V \Sigma^T U^T = V \Sigma U^T$; $(V \Sigma U^T)(U \Sigma V^T) = V \Sigma U^T U \Sigma V^T = V \Sigma^2 V^T$

II. cols are eigenvectors of $C = X_c^T X_c / (N-1)$ if the eigenval of C is $\lambda_i = \sigma_i^2 / (N-1)$; $C = \frac{1}{N-1} X_c^T X_c = \frac{1}{N-1} V \Sigma^2 V^T$

$$Cv = \frac{1}{N-1} V \Sigma^2 V^T V = \frac{1}{N-1} V \Sigma^2 e_i = \frac{\sigma_i^2}{N-1} V e_i = \frac{\sigma_i^2}{N-1} v_i \quad \text{each col of } V \text{ is eigenvector of } C$$

III. SVD Preferred when d is large? C is $d \times d = 2 \times k \times 2 \times k$ very high mem consumption

Svd works directly on x so 801×20531 much smaller, when $N \ll d$ so 156×20531

Here are at most $N-1$ non-zero eigenvalues anyway so you're not losing anything by using SVD on smaller matrix

Problem 3: Penguin Species Discovery with K-Means

This notebook implements k-means from scratch in NumPy and applies it to the Palmer Penguins morphology features in `penguins.csv`. Species labels are held out during clustering and used only afterward for interpretation.

(a) Exploratory Data Analysis

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.metrics import silhouette_score

sns.set_theme(style="whitegrid", context="notebook")
plt.rcParams["figure.figsize"] = (8, 5)

DATA_PATH = "penguins.csv"
FEATURES = ["bill_length_mm", "bill_depth_mm", "flipper_length_mm", "body_mass_g"]

penguins = pd.read_csv(DATA_PATH, na_values=["NA"])

print("Shape:", penguins.shape)
display(penguins.describe(include="all"))

print("Penguins per species:")
display(penguins["species"].value_counts().rename("count").to_frame())

print("Penguins per island:")
display(penguins["island"].value_counts().rename("count").to_frame())

print("Missing values per column:")
missing_by_column = penguins.isna().sum().rename("missing_count").to_frame()
display(missing_by_column)

clean_penguins = penguins.dropna(subset=FEATURES).copy()
print(f"Rows remaining after dropping NaN in morphology features: {len(clean_penguins)}")

plt.figure(figsize=(8, 5.5))
sns.scatterplot(
    data=clean_penguins,
    x="flipper_length_mm",
    y="bill_length_mm",
    hue="species",
    style="species",
    s=70,
    edgecolor="white",
```

```
linewidth=0.5,
)
plt.title("Penguins by Recorded Species")
plt.xlabel("Flipper length (mm)")
plt.ylabel("Bill length (mm)")
plt.tight_layout()
plt.show()
```

Shape: (344, 8)

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	s
count	344	344	342.000000	342.000000	342.000000	342.000000	3
unique	3	3	NaN	NaN	NaN	NaN	
top	Adelie	Biscoe	NaN	NaN	NaN	NaN	m
freq	152	168	NaN	NaN	NaN	NaN	1
mean	NaN	NaN	43.921930	17.151170	200.915205	4201.754386	N
std	NaN	NaN	5.459584	1.974793	14.061714	801.954536	N
min	NaN	NaN	32.100000	13.100000	172.000000	2700.000000	N
25%	NaN	NaN	39.225000	15.600000	190.000000	3550.000000	N
50%	NaN	NaN	44.450000	17.300000	197.000000	4050.000000	N
75%	NaN	NaN	48.500000	18.700000	213.000000	4750.000000	N
max	NaN	NaN	59.600000	21.500000	231.000000	6300.000000	N



Penguins per species:

count	
species	
Adelie	152
Gentoo	124
Chinstrap	68

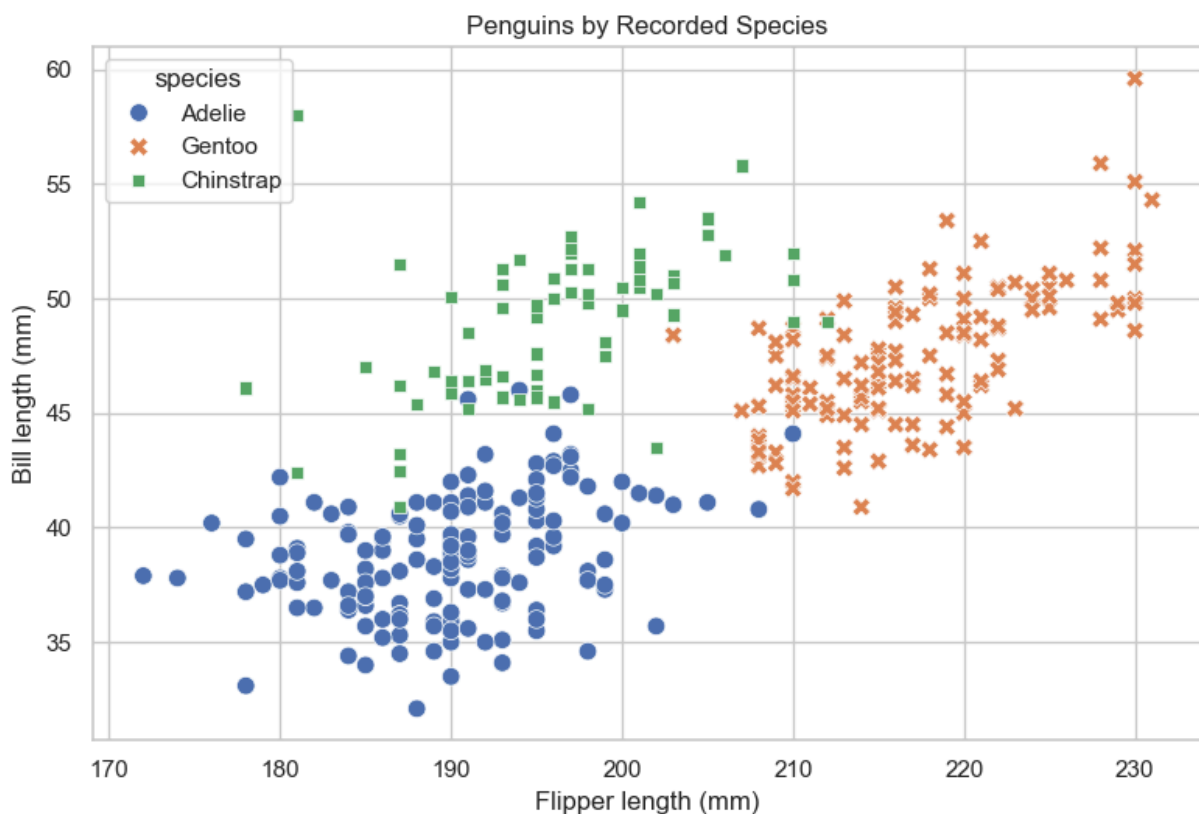
Penguins per island:

count	
island	
Biscoe	168
Dream	124
Torgersen	52

Missing values per column:

missing_count	
species	0
island	0
bill_length_mm	2
bill_depth_mm	2
flipper_length_mm	2
body_mass_g	2
sex	11
year	0

Rows remaining after dropping NaN in morphology features: 342



There is a clear Gentoo blob with longer flippers, while Adelie and Chinstrap overlap more in this two-dimensional projection. Chinstrap tends to have a longer bill than Adelie, but those two species are the most similar by eye in the flipper-length vs. bill-length plot.

(b) Preprocessing

```
In [2]: X_original_df = clean_penguins[FEATURES].astype(float).copy()
true_species = clean_penguins["species"].copy()

feature_means = X_original_df.mean(axis=0)
```

```

feature_stds = X_original_df.std(axis=0, ddof=0)
X = ((X_original_df - feature_means) / feature_stds).to_numpy()

scaled_check = pd.DataFrame(
    {
        "scaled_mean": X.mean(axis=0),
        "scaled_std": X.std(axis=0, ddof=0),
    },
    index=FEATURES,
)
display(scaled_check)

print("Original feature means used for standardization:")
display(feature_means.rename("mean").to_frame())
print("Original feature standard deviations used for standardization:")
display(feature_stds.rename("std").to_frame())

```

	scaled_mean	scaled_std
bill_length_mm	1.662088e-16	1.0
bill_depth_mm	4.155221e-16	1.0
flipper_length_mm	-8.310441e-16	1.0
body_mass_g	8.310441e-17	1.0

Original feature means used for standardization:

	mean
bill_length_mm	43.921930
bill_depth_mm	17.151170
flipper_length_mm	200.915205
body_mass_g	4201.754386

Original feature standard deviations used for standardization:

	std
bill_length_mm	5.451596
bill_depth_mm	1.971904
flipper_length_mm	14.041141
body_mass_g	800.781229

The four clustering features are `bill_length_mm`, `bill_depth_mm`, `flipper_length_mm`, and `body_mass_g`; categorical columns and the held-out `species` label are excluded. Standardization is necessary because k-means uses Euclidean distance: without scaling, `body_mass_g` would dominate simply because it is measured in thousands, while bill measurements are much smaller numeric values.

(c) Implement K-Means from Scratch

```
In [3]: def _kmeans_internal(X, k, max_iter=100, seed=0, return_history=False):
    X = np.asarray(X, dtype=float)
    n_samples = X.shape[0]
    if k < 1 or k > n_samples:
        raise ValueError("k must be between 1 and the number of samples")

    rng = np.random.default_rng(seed)
    initial_idx = rng.choice(n_samples, size=k, replace=False)
    centroids = X[initial_idx].copy()
    labels = np.full(n_samples, -1, dtype=int)
    inertia_history = []

    for _ in range(max_iter):
        distances_sq = ((X[:, None, :] - centroids[None, :, :]) ** 2).sum(axis=2)
        new_labels = np.argmin(distances_sq, axis=1)

        new_centroids = centroids.copy()
        for cluster_id in range(k):
            members = X[new_labels == cluster_id]
            if len(members) > 0:
                new_centroids[cluster_id] = members.mean(axis=0)
            else:
                new_centroids[cluster_id] = X[rng.integers(n_samples)]

        updated_distances_sq = ((X - new_centroids[new_labels]) ** 2).sum(axis=1)
        inertia_history.append(float(updated_distances_sq.sum()))

        if np.array_equal(new_labels, labels):
            centroids = new_centroids
            labels = new_labels
            break

        centroids = new_centroids
        labels = new_labels

    final_distances_sq = ((X[:, None, :] - centroids[None, :, :]) ** 2).sum(axis=2)
    final_labels = np.argmin(final_distances_sq, axis=1)
    final_inertia = float(final_distances_sq[np.arange(n_samples), final_labels].sum())

    if not inertia_history or not np.isclose(inertia_history[-1], final_inertia):
        inertia_history.append(final_inertia)

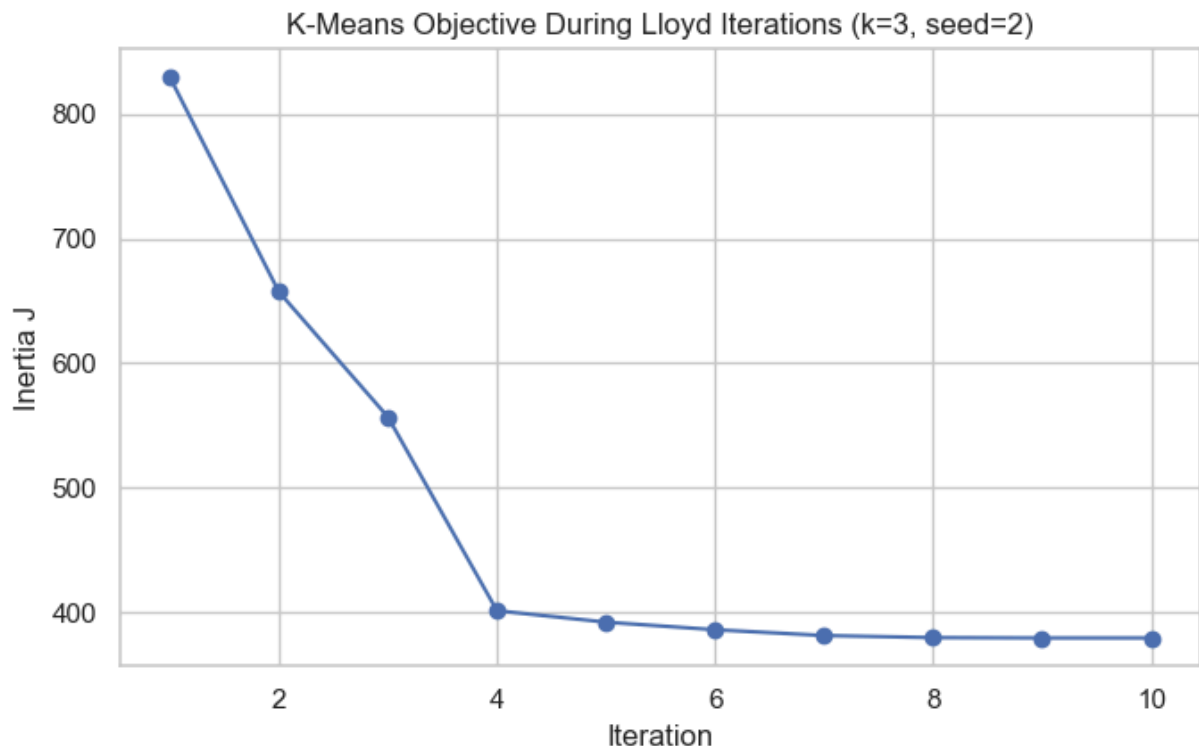
    if return_history:
        return centroids, final_labels, final_inertia, inertia_history
    return centroids, final_labels, final_inertia

def kmeans(X, k, max_iter=100, seed=0):
    # Required assignment signature.
    return _kmeans_internal(X, k, max_iter=max_iter, seed=seed, return_history=False)
```

```
centroids_demo, labels_demo, inertia_demo, history_demo = _kmeans_internal(
    X, k=3, max_iter=100, seed=2, return_history=True
)

plt.figure(figsize=(7, 4.5))
plt.plot(range(1, len(history_demo) + 1), history_demo, marker="o")
plt.title("K-Means Objective During Lloyd Iterations (k=3, seed=2)")
plt.xlabel("Iteration")
plt.ylabel("Inertia J")
plt.tight_layout()
plt.show()

print("Inertia history:", np.round(history_demo, 4))
print("Monotone non-increasing:", bool(np.all(np.diff(history_demo) <= 1e-10)))
```



```
Inertia history: [829.5432 657.2344 556.8323 401.2819 392.0808 386.0062 381.349 379.6756
 379.3925 379.3925]
Monotone non-increasing: True
```

The plotted objective is monotone non-increasing, as expected from Lloyd's assignment and centroid-update steps.

(d) Cluster and Validate

```
In [4]: main_seed = 2
centroids_k3, labels_k3, inertia_k3 = kmeans(X, k=3, seed=main_seed)
print(f"Final inertia for k=3 with seed={main_seed}: {inertia_k3:.3f}")

cluster_species_table = pd.crosstab(
    pd.Series(labels_k3, name="cluster"),
```

```

    true_species.rename("species"),
)
display(cluster_species_table)

purity = cluster_species_table.max(axis=1).sum() / cluster_species_table.to_numpy()
print(f"Clustering purity: {purity:.3f}")

centroids_original = centroids_k3 * feature_stds.to_numpy() + feature_means.to_numpy()
centroid_table = pd.DataFrame(centroids_original, columns=FEATURES)
centroid_table.index.name = "cluster"
display(centroid_table.round(2))

```

Final inertia for k=3 with seed=2: 379.393

species Adelie Chinstrap Gentoo

cluster

0	24	61	2
1	126	5	0
2	1	0	121

Clustering purity: 0.906

bill_length_mm bill_depth_mm flipper_length_mm body_mass_g

cluster

0	47.53	18.76	196.90	3902.01
1	38.21	18.11	188.40	3584.66
2	47.50	14.98	217.19	5076.02

Cluster 0 is a Chinstrap-like group: long bills, deep bills, moderate flippers, and medium body mass. Cluster 1 is an Adelie-like group: shorter bills, deep bills, shorter flippers, and smaller body mass. Cluster 2 is a Gentoo-like group: large body mass, long flippers, and comparatively shallow bills.

In [5]: `plot_df = clean_penguins.assign(cluster=labels_k3.astype(str))`

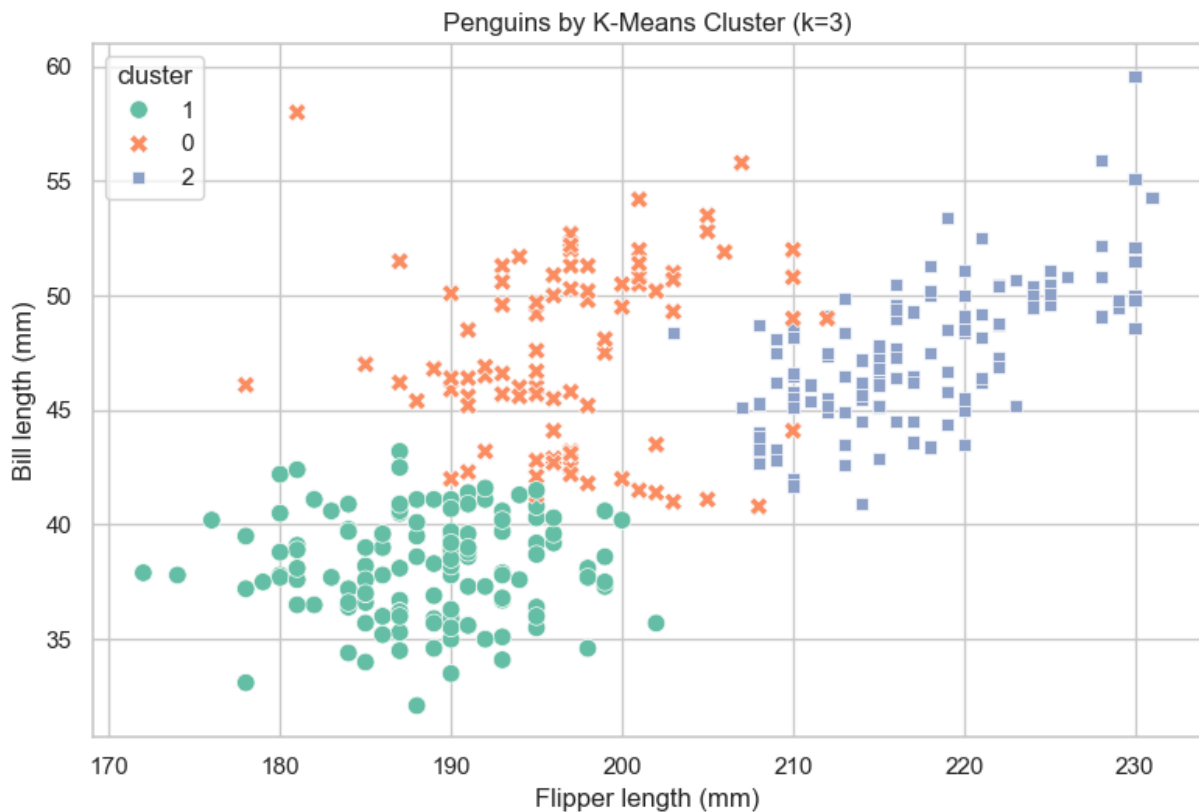
```

plt.figure(figsize=(8, 5.5))
sns.scatterplot(
    data=plot_df,
    x="flipper_length_mm",
    y="bill_length_mm",
    hue="cluster",
    style="cluster",
    palette="Set2",
    s=70,
    edgecolor="white",
    linewidth=0.5,
)
plt.title("Penguins by K-Means Cluster (k=3)")
plt.xlabel("Flipper length (mm)")

```



```
plt.ylabel("Bill length (mm)")
plt.tight_layout()
plt.show()
```



The cluster-colored scatter mostly resembles the species-colored version: the Gentoo-like group is cleanly separated, while Adelie and Chinstrap overlap in the center-left of the plot. The main differences occur along the Adelie/Chinstrap boundary, where morphology is less cleanly separated in these two dimensions.

(e) The Elbow Method

```
In [6]: elbow_rows = []
for k in range(1, 11):
    inertias = [kmeans(X, k=k, seed=seed)[2] for seed in range(10)]
    elbow_rows.append(
        {
            "k": k,
            "mean_inertia": float(np.mean(inertias)),
            "std_inertia": float(np.std(inertias)),
            "best_inertia": float(np.min(inertias)),
        }
    )

elbow_df = pd.DataFrame(elbow_rows)
display(elbow_df.round(3))

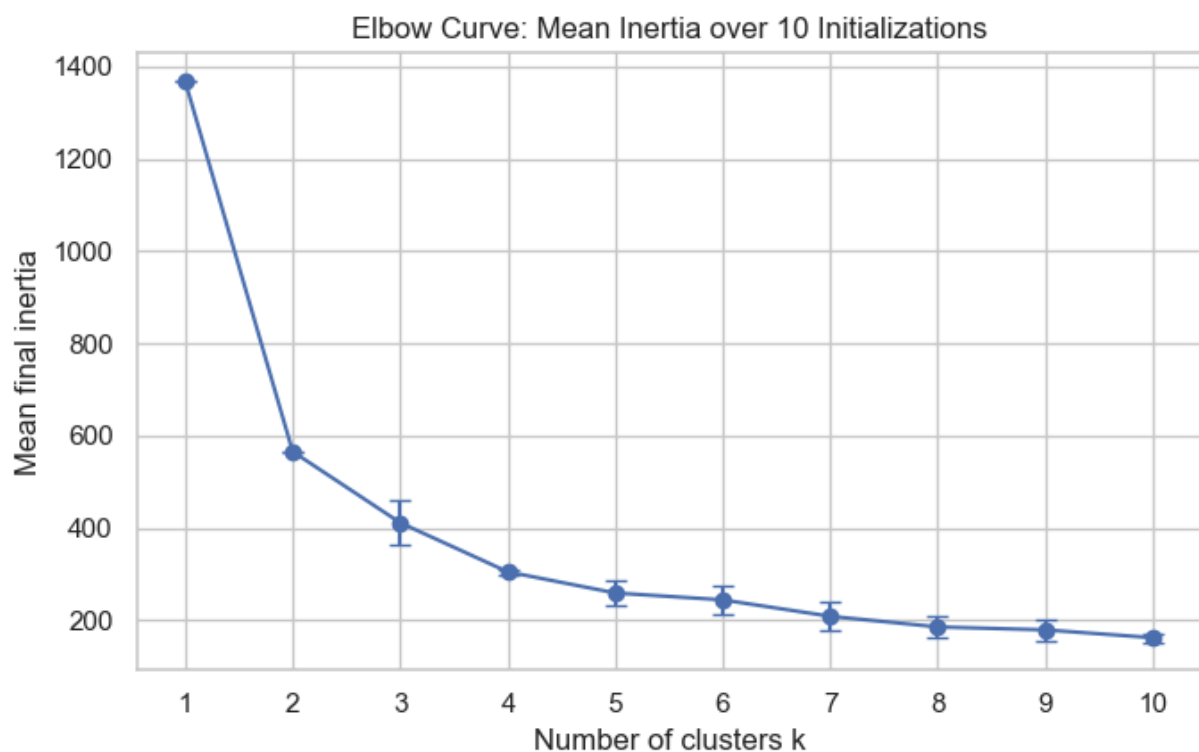
plt.figure(figsize=(7, 4.5))
plt.errorbar(
```

```

    elbow_df["k"],
    elbow_df["mean_inertia"],
    yerr=elbow_df["std_inertia"],
    marker="o",
    capsize=4,
)
plt.xticks(range(1, 11))
plt.title("Elbow Curve: Mean Inertia over 10 Initializations")
plt.xlabel("Number of clusters k")
plt.ylabel("Mean final inertia")
plt.tight_layout()
plt.show()

```

	k	mean_inertia	std_inertia	best_inertia
0	1	1368.000	0.000	1368.000
1	2	565.708	0.000	565.708
2	3	411.627	48.865	379.393
3	4	305.269	5.222	300.400
4	5	259.483	27.127	232.597
5	6	244.537	30.338	205.127
6	7	209.003	30.523	187.155
7	8	185.960	22.712	171.368
8	9	179.548	23.879	160.739
9	10	162.613	9.241	152.667



The largest improvement is from `k=1` to `k=2`, and a smaller but still meaningful bend appears around `k=3`. This is partly consistent with the three recorded species, but it also reveals that the strongest morphology split is Gentoo versus non-Gentoo, while Adelie and Chinstrap are harder to separate.

(f) Silhouette Score

```
In [7]: silhouette_rows = []
best_runs_by_k = {}

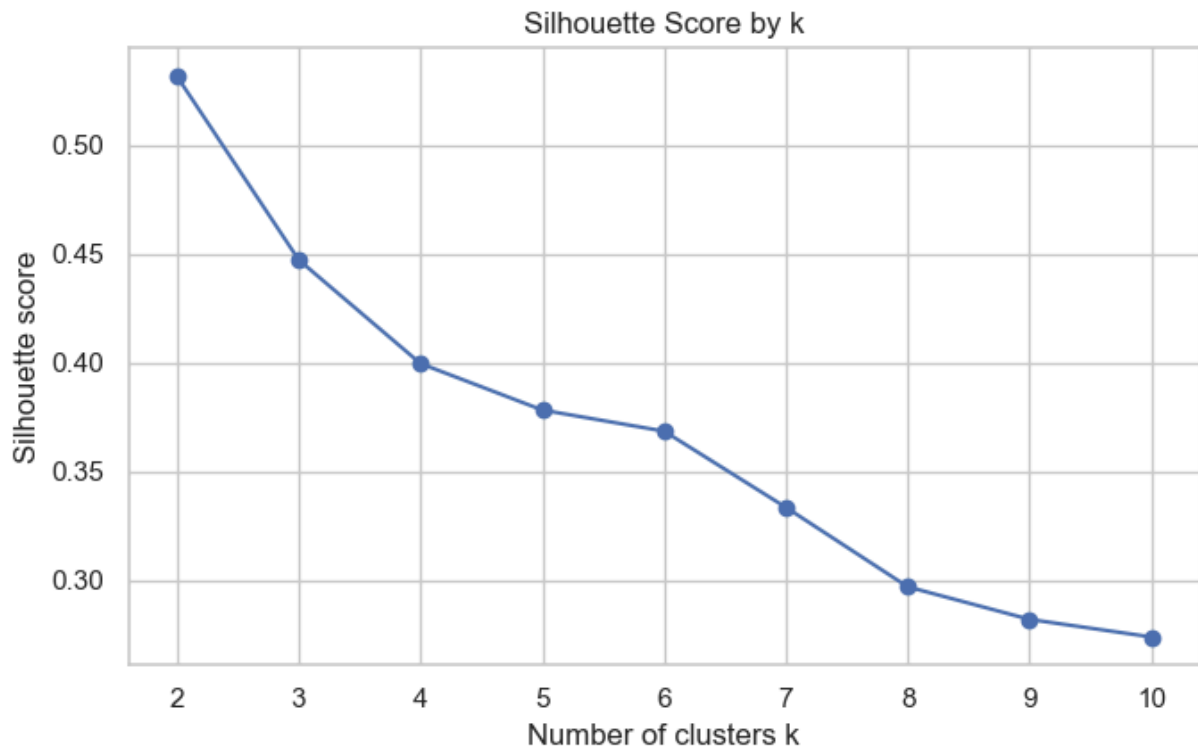
for k in range(2, 11):
    runs = [kmeans(X, k=k, seed=seed) for seed in range(10)]
    best_centroids, best_labels, best_inertia = min(runs, key=lambda run: run[2])
    score = silhouette_score(X, best_labels)
    best_runs_by_k[k] = (best_centroids, best_labels, best_inertia)
    silhouette_rows.append({"k": k, "silhouette": score, "best_inertia": best_inertia})

silhouette_df = pd.DataFrame(silhouette_rows)
display(silhouette_df.round(3))

plt.figure(figsize=(7, 4.5))
plt.plot(silhouette_df["k"], silhouette_df["silhouette"], marker="o")
plt.xticks(range(2, 11))
plt.title("Silhouette Score by k")
plt.xlabel("Number of clusters k")
plt.ylabel("Silhouette score")
plt.tight_layout()
plt.show()

best_silhouette_row = silhouette_df.loc[silhouette_df["silhouette"].idxmax()]
print(f"Best silhouette k: {int(best_silhouette_row['k'])} with score {best_silhouette_row['silhouette']}")
```

	k	silhouette	best_inertia
0	2	0.532	565.708
1	3	0.447	379.393
2	4	0.400	300.400
3	5	0.378	232.597
4	6	0.369	205.127
5	7	0.334	187.155
6	8	0.297	171.368
7	9	0.282	160.739
8	10	0.274	152.667



Best silhouette k: 2 with score 0.532

Silhouette is maximized at `k=2`, which agrees with the strongest elbow bend but not with the three recorded species. I would trust silhouette more for the strictly unsupervised question of how many compact, well-separated morphology groups are present; I would still inspect `k=3` because the biological labels tell us there are three species and the third cluster captures meaningful Adelie/Chinstrap structure.

(g) Sensitivity to Initialization

```
In [8]: seed_rows = []
for seed in range(50):
    centroids_seed, labels_seed, inertia_seed = kmeans(X, k=3, seed=seed)
    seed_rows.append(
        {
            "seed": seed,
            "inertia": inertia_seed,
            "centroids": centroids_seed,
            "labels": labels_seed,
        }
    )

seed_df = pd.DataFrame(seed_rows)
best_seed_row = seed_df.loc[seed_df["inertia"].idxmin()]
worst_seed_row = seed_df.loc[seed_df["inertia"].idxmax()]
inertia_spread = seed_df["inertia"].max() - seed_df["inertia"].min()

print(f"Lowest inertia: {best_seed_row['inertia']:.3f} at seed {int(best_seed_row['seed'])}")
print(f"Highest inertia: {worst_seed_row['inertia']:.3f} at seed {int(worst_seed_row['seed'])}")
print(f"Spread: {inertia_spread:.3f}")
```

```

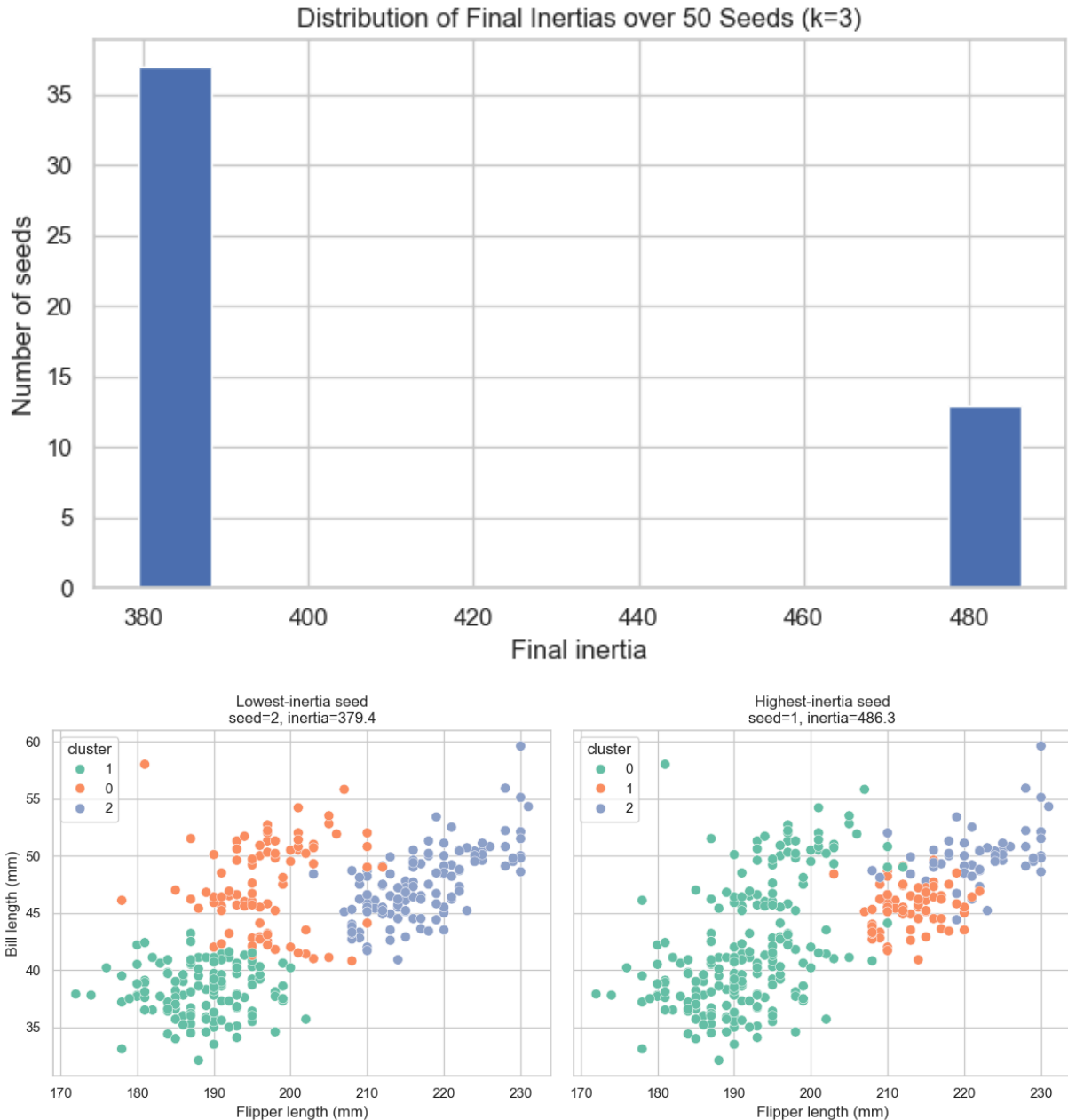
plt.figure(figsize=(7, 4.5))
plt.hist(seed_df["inertia"], bins=12, edgecolor="white")
plt.title("Distribution of Final Inertias over 50 Seeds (k=3)")
plt.xlabel("Final inertia")
plt.ylabel("Number of seeds")
plt.tight_layout()
plt.show()

fig, axes = plt.subplots(1, 2, figsize=(12, 5), sharex=True, sharey=True)
for ax, row, title in [
    (axes[0], best_seed_row, "Lowest-inertia seed"),
    (axes[1], worst_seed_row, "Highest-inertia seed"),
]:
    temp_df = clean_penguins.assign(cluster=row["labels"].astype(str))
    sns.scatterplot(
        data=temp_df,
        x="flipper_length_mm",
        y="bill_length_mm",
        hue="cluster",
        palette="Set2",
        s=60,
        edgecolor="white",
        linewidth=0.4,
        ax=ax,
        legend=True,
    )
    ax.set_title(f"{title}\nseed={int(row['seed'])}, inertia={row['inertia']:.1f}")
    ax.set_xlabel("Flipper length (mm)")
    ax.set_ylabel("Bill length (mm)")

plt.tight_layout()
plt.show()

```

Lowest inertia: 379.393 at seed 2
 Highest inertia: 486.297 at seed 1
 Spread: 106.905



The spread is large enough to matter: some seeds find a strong three-species-like solution, while others split the Gentoo-like birds into two clusters and merge much of Adelie/Chinstrap. The two side-by-side clusterings therefore disagree substantially, not just on a few boundary points.

(h) Summary and Recommendation

The morphology data show a very strong split into two compact groups, with Gentoo separated from the other penguins, and a plausible but less clean three-cluster structure corresponding to the three recorded species. The elbow curve has its strongest bend at $k=2$ and a secondary bend around $k=3$, while silhouette selects $k=2$, so I am most confident in two broad morphology groups and moderately confident in a species-level $k=3$ clustering. Using $k=3$, the best fixed run in this notebook recovered the recorded

species with purity around 0.91, with most errors occurring between Adelie and Chinstrap. Initialization matters: across 50 random seeds, Lloyd's algorithm landed in noticeably different local minima, so in practice I would run many initializations and keep the lowest-inertia solution. A limitation is that this analysis uses only four morphology measurements and drops rows with missing morphology values rather than modeling or imputing missingness.

Problem 4: PCA from Scratch vs. UMAP on Cancer Gene Expression

This notebook implements every part of Problem 4 using the local

`gene+expression+cancer+rna+seq.zip` archive. PCA is implemented from scratch in NumPy using both SVD and the Gram-matrix eigendecomposition trick. UMAP uses the `umap-learn` package, as allowed by the prompt.

```
In [12]: from pathlib import Path
import io
import tarfile
import zipfile
import warnings

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split

plt.style.use("seaborn-v0_8-whitegrid")
RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
```

4(a) Look at the data

```
In [13]: def read_gene_expression_archive(zip_path="gene+expression+cancer+rna+seq.zip"):
    """Load data.csv and labels.csv from the UCI zip without manual extraction."""
    zip_path = Path(zip_path)
    extracted_dir = Path("TCGA-PANCAN-HiSeq-801x20531")
    extracted_data = extracted_dir / "data.csv"
    extracted_labels = extracted_dir / "labels.csv"

    if extracted_data.exists() and extracted_labels.exists():
        return pd.read_csv(extracted_data), pd.read_csv(extracted_labels)

    with zipfile.ZipFile(zip_path) as zf:
        tar_name = next(name for name in zf.namelist() if name.endswith(".tar.gz"))
        tar_bytes = io.BytesIO(zf.read(tar_name))

        with tarfile.open(fileobj=tar_bytes, mode="r:gz") as tf:
            members = {member.name: member for member in tf.getmembers()}
            data_name = next(name for name in members if name.endswith("/data.csv"))
            labels_name = next(name for name in members if name.endswith("/labels.csv"))

            with tf.extractfile(members[data_name]) as data_file:
```

```

        data = pd.read_csv(data_file)
        with tf.extractfile(members[labels_name]) as labels_file:
            labels = pd.read_csv(labels_file)

    return data, labels

data_raw, labels_raw = read_gene_expression_archive()
print("data.csv shape:", data_raw.shape)
print("labels.csv shape:", labels_raw.shape)
print("data columns, first 5:", list(data_raw.columns[:5]))
print("labels columns:", list(labels_raw.columns))

gene_id_col = data_raw.columns[0]
label_id_col = labels_raw.columns[0]
label_col = labels_raw.columns[1]

X_df = data_raw.drop(columns=[gene_id_col])
y = labels_raw[label_col].astype(str)

print("X shape after dropping ID:", X_df.shape)
print("Class counts:")
display(y.value_counts().sort_index())

```

```

data.csv shape: (801, 20532)
labels.csv shape: (801, 2)
data columns, first 5: ['Unnamed: 0', 'gene_0', 'gene_1', 'gene_2', 'gene_3']
labels columns: ['Unnamed: 0', 'Class']
X shape after dropping ID: (801, 20531)
Class counts:
Class
BRCA    300
COAD     78
KIRC    146
LUAD    141
PRAD    136
Name: count, dtype: int64

```

```

In [14]: assert X_df.shape == (801, 20531), f"Expected (801, 20531), got {X_df.shape}"
         assert len(y) == len(X_df), "Labels and data must have the same number of rows."

```

```

In [15]: # Full describe is very wide, so also show a reproducible subset of gene columns.
         random_gene_cols = X_df.sample(n=8, axis=1, random_state=RANDOM_STATE).columns
         print("Summary for a few random gene columns:")
         display(X_df[random_gene_cols].describe())

         value_min = X_df.to_numpy().min()
         value_max = X_df.to_numpy().max()
         print(f"Approximate expression-value range in this dataset: {value_min:.3f} to {val

```

Summary for a few random gene columns:

	gene_1272	gene_3222	gene_6423	gene_16755	gene_12457	gene_6956	gene_5921
count	801.000000	801.000000	801.000000	801.000000	801.000000	801.000000	801.000000
mean	11.063668	6.614294	10.556457	0.278427	0.010162	7.995829	6.716145
std	1.051581	2.938234	0.549155	0.564358	0.092842	1.192065	0.457912
min	5.064111	0.000000	8.978241	0.000000	0.000000	3.006711	4.689584
25%	10.505047	4.502872	10.170013	0.000000	0.000000	7.343052	6.429167
50%	11.172234	6.300005	10.500324	0.000000	0.000000	8.064150	6.724732
75%	11.677772	8.917253	10.899795	0.416083	0.000000	8.743064	6.985762
max	14.292135	14.245501	12.795739	3.828093	1.725305	11.635110	8.642691

Approximate expression-value range in this dataset: 0.000 to 20.779

The expression values are already on a normalized log-like scale, so I do not apply an additional log transform. I still standardize gene columns before PCA/UMAP so each gene contributes comparably; otherwise high-variance genes would dominate Euclidean distances and the PCA objective.

```
In [16]: gene_means = X_df.mean(axis=0)
gene_stds = X_df.std(axis=0, ddof=0).replace(0, 1.0)
X_std_df = (X_df - gene_means) / gene_stds
X_std = X_std_df.to_numpy(dtype=np.float64, copy=False)

print("Standardized mean range:", X_std_df.mean(axis=0).min(), "to", X_std_df.mean(
print("Standardized std range:", X_std_df.std(axis=0, ddof=0).min(), "to", X_std_df
```

Standardized mean range: -7.746335131117447e-15 to 7.826171393562402e-15
Standardized std range: 0.0 to 1.0000000000000007

4(b) PCA via SVD

```
In [17]: def pca_svd(X, k):
        """
        PCA from scratch using the SVD of centered data.

        Returns:
            components: shape (k, d), rows are principal axes
            projected: shape (N, k), centered data projected onto components
            explained_variance: shape (k,), sample variance explained by each component
        """
        X = np.asarray(X, dtype=np.float64)
        n_samples = X.shape[0]
        if not 1 <= k <= min(X.shape):
            raise ValueError("k must be between 1 and min(N, d).")

        X_centered = X - X.mean(axis=0)
        U, singular_values, Vt = np.linalg.svd(X_centered, full_matrices=False)
```

```

components = Vt[:k]
projected = X_centered @ components.T
explained_variance = (singular_values[:k] ** 2) / (n_samples - 1)
return components, projected, explained_variance

components_svd_2, pca2_svd, explained_svd_2 = pca_svd(X_std, k=2)
print("Explained variances for PC1 and PC2:", explained_svd_2)

```

Explained variances for PC1 and PC2: [2138.4510195 1776.17511467]

4(c) PCA via covariance eigendecomposition

The direct covariance matrix would be 20531×20531 , or about 3.4 GB as float64 values before any eigendecomposition overhead. Since $N \ll d$, the Gram matrix $X_c X_c.T / (N - 1)$ is only 801×801 , and its nonzero eigenvalues match the nonzero covariance eigenvalues.

```

In [18]: def pca_eig(X, k):
    """
    PCA from scratch using the small Gram matrix eigendecomposition.

    This avoids materializing the d x d covariance matrix when d is very large.
    """
    X = np.asarray(X, dtype=np.float64)
    n_samples = X.shape[0]
    if not 1 <= k <= min(X.shape):
        raise ValueError("k must be between 1 and min(N, d).")

    X_centered = X - X.mean(axis=0)
    gram = (X_centered @ X_centered.T) / (n_samples - 1)

    eigenvalues, U = np.linalg.eigh(gram)
    order = np.argsort(eigenvalues)[::-1]
    eigenvalues = eigenvalues[order]
    U = U[:, order]

    components = []
    for j in range(k):
        eigenvalue = max(eigenvalues[j], 0.0)
        if eigenvalue <= 1e-12:
            v = np.zeros(X.shape[1])
        else:
            v = (X_centered.T @ U[:, j]) / np.sqrt((n_samples - 1) * eigenvalue)
            v = v / np.linalg.norm(v)
        components.append(v)

    components = np.vstack(components)
    projected = X_centered @ components.T
    explained_variance = eigenvalues[:k]
    return components, projected, explained_variance

```

```

components_eig_2, pca2_eig, explained_eig_2 = pca_eig(X_std, k=2)
print("SVD explained variances:", explained_svd_2)
print("Eigen explained variances:", explained_eig_2)
print("Variances agree:", np.allclose(explained_svd_2, explained_eig_2, rtol=1e-8,

axis_abs_dot = np.abs(np.sum(components_svd_2 * components_eig_2, axis=1))
print("Absolute dot product between matching axes:", axis_abs_dot)
print("Axes agree up to sign:", np.allclose(axis_abs_dot, np.ones_like(axis_abs_dot)

```

SVD explained variances: [2138.4510195 1776.17511467]

Eigen explained variances: [2138.4510195 1776.17511467]

Variances agree: True

Absolute dot product between matching axes: [1. 1.]

Axes agree up to sign: True

The explained variances should match up to floating-point error, and the absolute dot products should be essentially 1.0. The absolute value is necessary because eigenvectors are defined only up to sign.

4(d) Scree plot

```

In [19]: max_components = min(X_std.shape) - 1
components_all, projected_all, explained_all = pca_svd(X_std, k=max_components)

variance_fraction = explained_all / explained_all.sum()
cumulative_fraction = np.cumsum(variance_fraction)
components_for_80 = int(np.searchsorted(cumulative_fraction, 0.80) + 1)

print(f"Computed PCA components: {max_components}")
print(f"Components needed for 80% variance: {components_for_80}")
print(f"Original number of genes: {X_std.shape[1]}")

plot_k = min(100, len(explained_all))
fig, axes = plt.subplots(1, 2, figsize=(13, 4))
axes[0].plot(np.arange(1, plot_k + 1), explained_all[:plot_k], marker="o", markersize=10)
axes[0].set_title("Scree plot")
axes[0].set_xlabel("Principal component")
axes[0].set_ylabel("Explained variance")

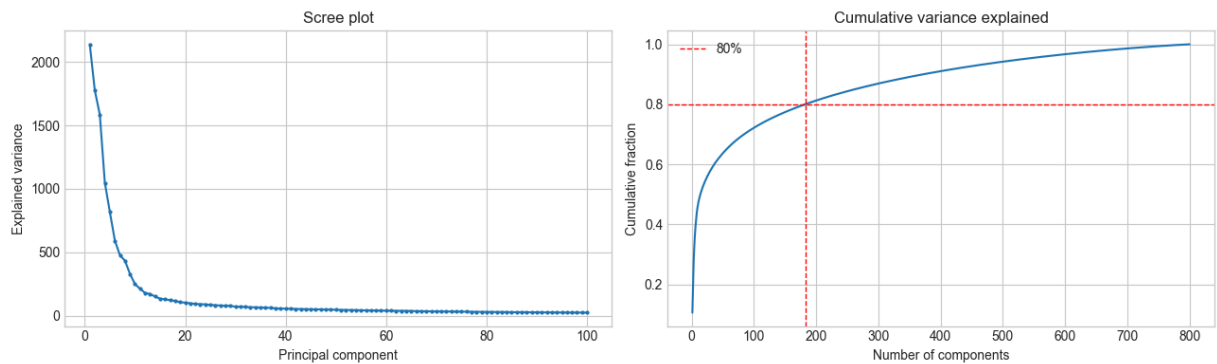
axes[1].plot(np.arange(1, len(cumulative_fraction) + 1), cumulative_fraction)
axes[1].axhline(0.80, color="red", linestyle="--", linewidth=1, label="80%")
axes[1].axvline(components_for_80, color="red", linestyle="--", linewidth=1)
axes[1].set_title("Cumulative variance explained")
axes[1].set_xlabel("Number of components")
axes[1].set_ylabel("Cumulative fraction")
axes[1].legend()
plt.tight_layout()

```

Computed PCA components: 800

Components needed for 80% variance: 183

Original number of genes: 20531

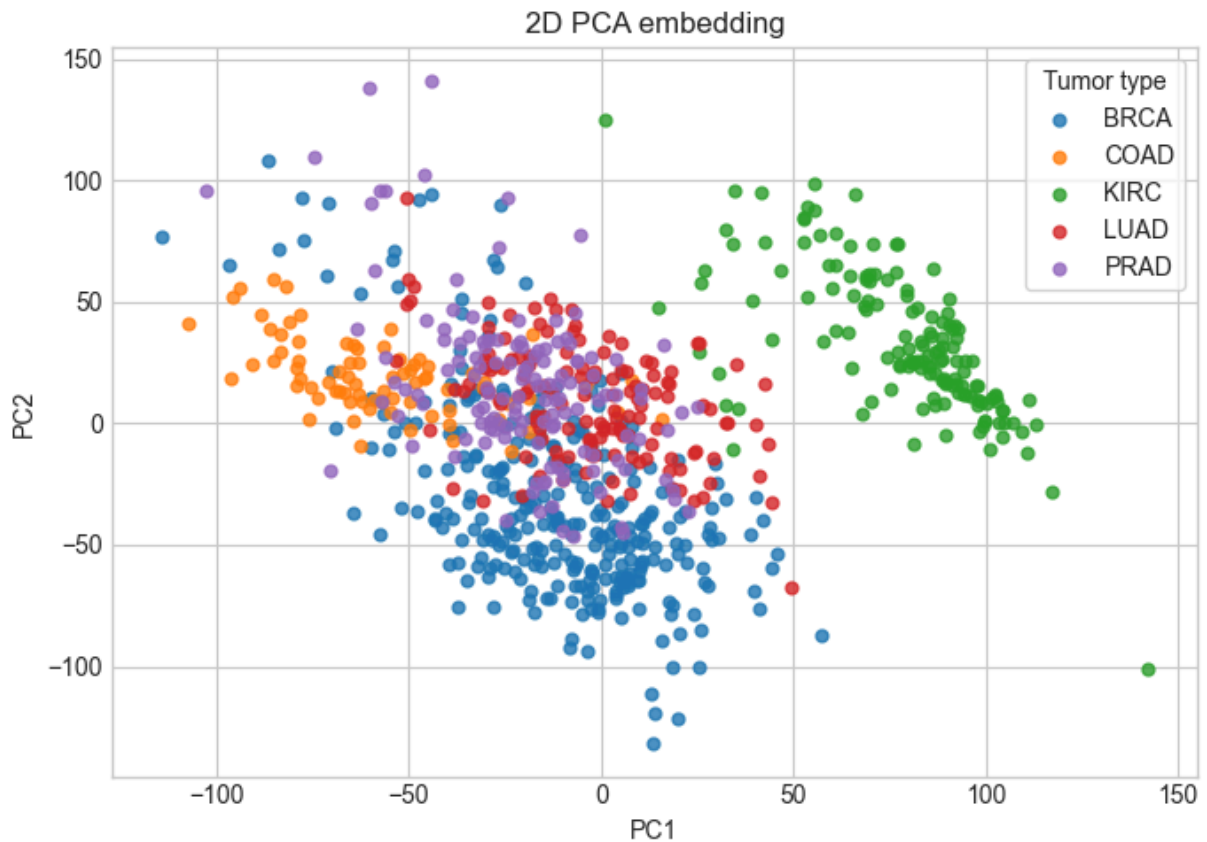


The number printed above is far smaller than the original 20,531 genes, showing that the dominant variation in these samples lives in a much lower-dimensional subspace.

4(e) 2D PCA visualization

```
In [20]: def scatter_embedding(embedding, labels, title, ax=None, xlabel="Dimension 1", ylabel="Dimension 2"):
    if ax is None:
        fig, ax = plt.subplots(figsize=(7, 5))
    classes = sorted(pd.Series(labels).unique())
    cmap = plt.get_cmap("tab10")
    for i, cls in enumerate(classes):
        mask = np.asarray(labels) == cls
        ax.scatter(embedding[mask, 0], embedding[mask, 1], s=24, alpha=0.8, color=cmap(cls))
    ax.set_title(title)
    ax.set_xlabel(xlabel)
    ax.set_ylabel(ylabel)
    ax.legend(title="Tumor type", frameon=True)
    return ax

pca_2d = projected_all[:, :2]
fig, ax = plt.subplots(figsize=(7, 5))
scatter_embedding(pca_2d, y.to_numpy(), "2D PCA embedding", ax=ax, xlabel="PC1", ylabel="PC2")
plt.tight_layout()
```



PCA should show meaningful tumor-type structure because tissue-specific expression patterns explain a large share of the variance. In a typical run on this dataset, several classes form clear regions, while the two adenocarcinoma classes can be closer or more partially mixed in a purely linear 2D projection.

4(f) What do PC1 and PC2 mean?

```
In [21]: gene_names = np.asarray(X_df.columns)

def top_loadings(component, gene_names, n=10):
    idx = np.argsort(np.abs(component))[:, -1][:n]
    return pd.DataFrame({
        "gene_index": idx,
        "gene": gene_names[idx],
        "loading": component[idx],
        "abs_loading": np.abs(component[idx]),
    })

pc1_top = top_loadings(components_all[0], gene_names, n=10)
pc2_top = top_loadings(components_all[1], gene_names, n=10)

print("Top absolute loadings for PC1:")
display(pc1_top)
print("Top absolute loadings for PC2:")
display(pc2_top)
```

```
pc1_genes = set(pc1_top["gene"])
pc2_genes = set(pc2_top["gene"])
print("Overlap among top-10 genes:", sorted(pc1_genes & pc2_genes))
```

Top absolute loadings for PC1:

	gene_index	gene	loading	abs_loading
0	19862	gene_19862	0.019002	0.019002
1	17360	gene_17360	0.018985	0.018985
2	13489	gene_13489	0.018966	0.018966
3	15158	gene_15158	0.018777	0.018777
4	7031	gene_7031	0.018740	0.018740
5	7019	gene_7019	0.018657	0.018657
6	10788	gene_10788	0.018629	0.018629
7	13507	gene_13507	-0.018624	0.018624
8	6543	gene_6543	0.018595	0.018595
9	2288	gene_2288	-0.018592	0.018592

Top absolute loadings for PC2:

	gene_index	gene	loading	abs_loading
0	3612	gene_3612	-0.020273	0.020273
1	1222	gene_1222	0.019863	0.019863
2	14699	gene_14699	-0.019862	0.019862
3	1010	gene_1010	0.019829	0.019829
4	19498	gene_19498	0.019505	0.019505
5	1113	gene_1113	-0.019409	0.019409
6	3603	gene_3603	-0.019346	0.019346
7	364	gene_364	-0.019225	0.019225
8	11153	gene_11153	0.019222	0.019222
9	19866	gene_19866	0.019133	0.019133

Overlap among top-10 genes: []

The top-loading genes are the genes that drive each principal direction most strongly. PC1 and PC2 usually have different top genes, which means they summarize different expression programs.

4(g) UMAP via library

If the next import fails, run this in a notebook cell first:

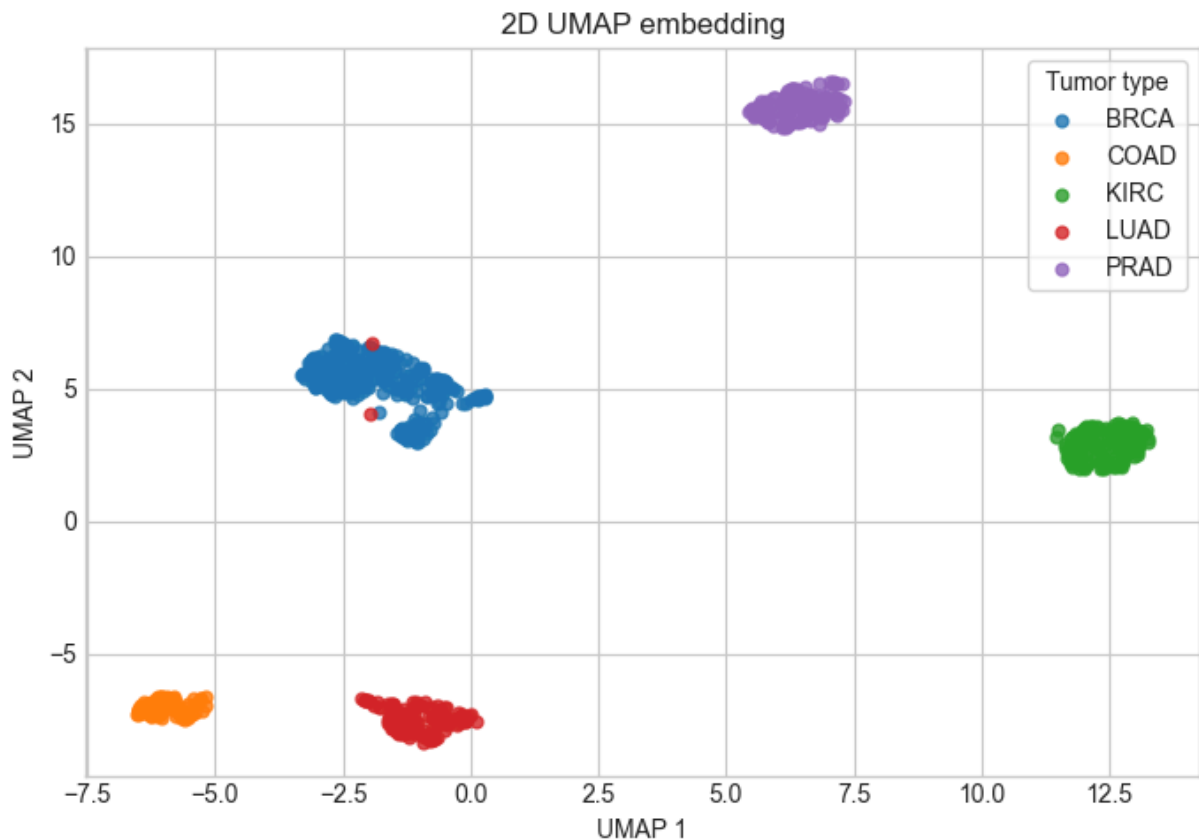
```
%pip install umap-learn
```

```
In [22]: try:
import umap.umap_ as umap
except ImportError as exc:
    raise ImportError("Install UMAP with `%pip install umap-learn`, then rerun this")

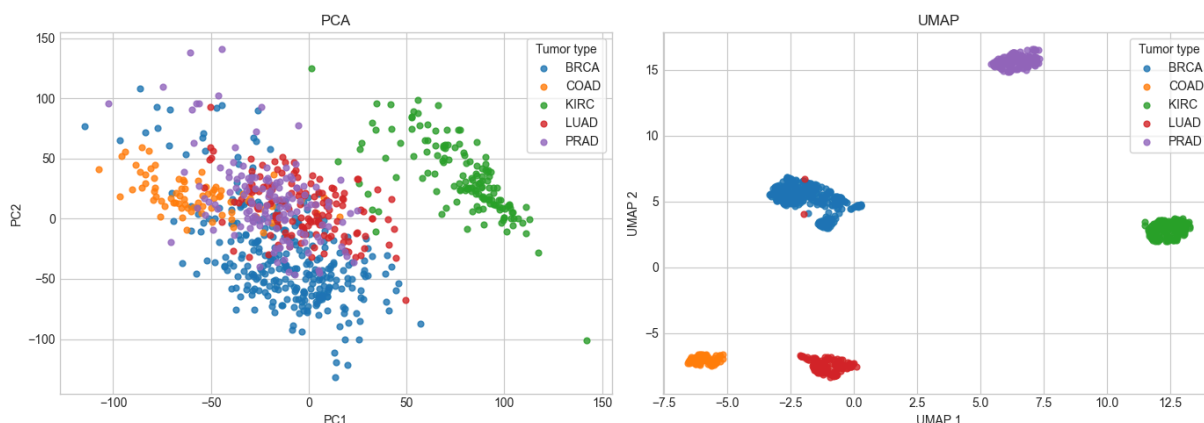
umap_model = umap.UMAP(n_components=2, random_state=RANDOM_STATE)
umap_2d = umap_model.fit_transform(X_std)

fig, ax = plt.subplots(figsize=(7, 5))
scatter_embedding(umap_2d, y.to_numpy(), "2D UMAP embedding", ax=ax, xlabel="UMAP 1",
plt.tight_layout()
```

C:\Users\HarryMyers\AppData\Roaming\Python\Python314\site-packages\tqdm\auto.py:21:
TqdmWarning: IProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
from .autonotebook import tqdm as notebook_tqdm
C:\Users\HarryMyers\AppData\Roaming\Python\Python314\site-packages\umap\umap_.py:195:
2: UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed for parallelism.
warn(



```
In [23]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))
scatter_embedding(pca_2d, y.to_numpy(), "PCA", ax=axes[0], xlabel="PC1", ylabel="PC2")
scatter_embedding(umap_2d, y.to_numpy(), "UMAP", ax=axes[1], xlabel="UMAP 1", ylabel="UMAP 2")
plt.tight_layout()
```



UMAP typically gives more visually separated groups than PCA because it can preserve nonlinear local neighborhoods instead of being restricted to one global linear projection. The tradeoff is that UMAP distances and axes are less directly interpretable than PCA loadings.

4(h) UMAP hyperparameter exploration

```
In [24]: neighbor_values = [5, 15, 50]
min_dist_values = [0.0, 0.5]

fig, axes = plt.subplots(len(min_dist_values), len(neighbor_values), figsize=(15, 8))
umap_grid_embeddings = {}

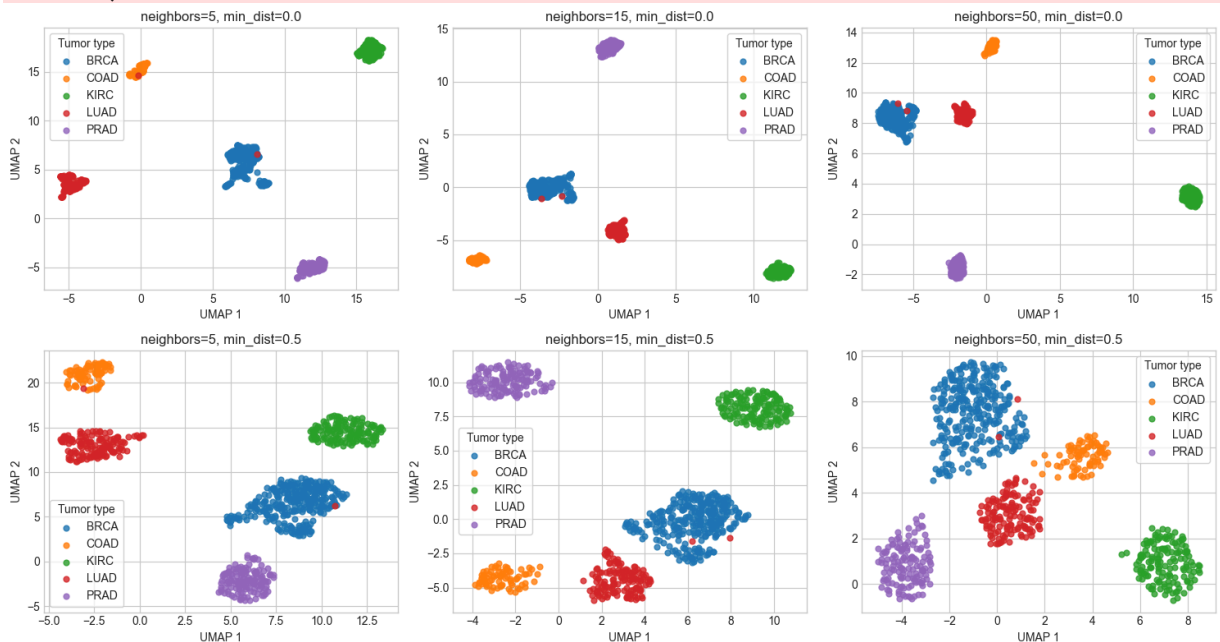
for row, min_dist in enumerate(min_dist_values):
    for col, n_neighbors in enumerate(neighbor_values):
        reducer = umap.UMAP(
            n_components=2,
            n_neighbors=n_neighbors,
            min_dist=min_dist,
            random_state=RANDOM_STATE,
        )
        emb = reducer.fit_transform(X_std)
        umap_grid_embeddings[(n_neighbors, min_dist)] = emb
        scatter_embedding(
            emb,
            y.to_numpy(),
            f"neighbors={n_neighbors}, min_dist={min_dist}",
            ax=axes[row, col],
            xlabel="UMAP 1",
            ylabel="UMAP 2",
        )

plt.tight_layout()
```

```

C:\Users\HarryMyers\AppData\Roaming\Python\Python314\site-packages\umap\umap_.py:195
2: UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
    warn(
C:\Users\HarryMyers\AppData\Roaming\Python\Python314\site-packages\umap\umap_.py:195
2: UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
    warn(
C:\Users\HarryMyers\AppData\Roaming\Python\Python314\site-packages\umap\umap_.py:195
2: UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
    warn(
C:\Users\HarryMyers\AppData\Roaming\Python\Python314\site-packages\umap\umap_.py:195
2: UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
    warn(
C:\Users\HarryMyers\AppData\Roaming\Python\Python314\site-packages\umap\umap_.py:195
2: UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
    warn(
C:\Users\HarryMyers\AppData\Roaming\Python\Python314\site-packages\umap\umap_.py:195
2: UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
    warn(

```



Smaller `n_neighbors` emphasizes very local structure, so clusters usually become tighter and more fragmented. Larger `n_neighbors` preserves more global structure, while larger `min_dist` spreads points out inside each group. For a lab-facing visualization, I would usually ship a middle or larger-neighborhood setting, such as `n_neighbors=15` or `50`, with `min_dist=0.0` if the goal is clean class separation.

4(i) Downstream task: does the embedding actually help?

```
In [25]: indices = np.arange(len(y))
train_idx, test_idx = train_test_split(
    indices,
    test_size=0.20,
    random_state=RANDOM_STATE,
    stratify=y,
)

X_train_raw = X_df.iloc[train_idx].to_numpy(dtype=np.float64, copy=True)
X_test_raw = X_df.iloc[test_idx].to_numpy(dtype=np.float64, copy=True)
y_train = y.iloc[train_idx].to_numpy()
y_test = y.iloc[test_idx].to_numpy()

train_means = X_train_raw.mean(axis=0)
train_stds = X_train_raw.std(axis=0, ddof=0)
train_stds[train_stds == 0] = 1.0
X_train_full = (X_train_raw - train_means) / train_stds
X_test_full = (X_test_raw - train_means) / train_stds

print("Train size:", X_train_full.shape[0], "Test size:", X_test_full.shape[0])
print("Train class counts:")
display(pd.Series(y_train).value_counts().sort_index())
print("Test class counts:")
display(pd.Series(y_test).value_counts().sort_index())
```

Train size: 640 Test size: 161

Train class counts:

BRCA	240
COAD	62
KIRC	116
LUAD	113
PRAD	109

Name: count, dtype: int64

Test class counts:

BRCA	60
COAD	16
KIRC	30
LUAD	28
PRAD	27

Name: count, dtype: int64

```
In [26]: def fit_logistic_and_score(X_train, X_test, y_train, y_test):
    clf = LogisticRegression(max_iter=2000, random_state=RANDOM_STATE)
    clf.fit(X_train, y_train)
    pred = clf.predict(X_test)
    return accuracy_score(y_test, pred), clf

# Full 20,531-gene baseline.
full_acc, full_clf = fit_logistic_and_score(X_train_full, X_test_full, y_train, y_test)

# PCA embedding fit on training data only, then applied to test data.
# Fit 10 components once so the next cell can also answer the reflection question.
pca_train_components_10, X_train_pca_10d, _ = pca_svd(X_train_full, k=10)
pca_train_center = X_train_full.mean(axis=0)
X_test_pca_10d = (X_test_full - pca_train_center) @ pca_train_components_10.T
X_train_pca_2d = X_train_pca_10d[:, :2]
```

```

X_test_pca_2d = X_test_pca_10d[:, :2]
pca_acc, pca_clf = fit_logistic_and_score(X_train_pca_2d, X_test_pca_2d, y_train, y_test)

# UMAP fit on training data only, then transformed for test data.
umap_train_model = umap.UMAP(n_components=2, random_state=RANDOM_STATE)
X_train_umap_2d = umap_train_model.fit_transform(X_train_full)
X_test_umap_2d = umap_train_model.transform(X_test_full)
umap_acc, umap_clf = fit_logistic_and_score(X_train_umap_2d, X_test_umap_2d, y_train, y_test)

accuracy_table = pd.DataFrame({
    "method": ["Full gene expression", "PCA", "UMAP"],
    "dimensions": [X_train_full.shape[1], 2, 2],
    "test_accuracy": [full_acc, pca_acc, umap_acc],
})
display(accuracy_table)

```

C:\Users\HarryMyers\AppData\Roaming\Python\Python314\site-packages\umap\umap_.py:195: UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed for parallelism.

```
warn(
```

	method	dimensions	test_accuracy
0	Full gene expression	20531	0.981366
1	PCA	2	0.633540
2	UMAP	2	0.981366

```

In [27]: # Optional: check whether 10 PCA components help relative to 2D PCA.
pca10_acc, pca10_clf = fit_logistic_and_score(X_train_pca_10d, X_test_pca_10d, y_train, y_test)
print(f"10D PCA logistic-regression test accuracy: {pca10_acc:.4f}")

```

10D PCA logistic-regression test accuracy: 0.9938

The full-gene classifier is the strongest baseline because it keeps all measured information. A 2D embedding is useful for visualization, but it is an aggressive compression; 10 PCA components often recover more predictive signal than 2 PCA components while still being far smaller than 20,531 genes.

4(j) Summary

```

In [28]: summary_table = accuracy_table.copy()
summary_table.loc[len(summary_table)] = ["PCA", 10, pca10_acc]
display(summary_table.sort_values(["dimensions", "method"]))

```


	method	dimensions	test_accuracy
1	PCA	2	0.633540
2	UMAP	2	0.981366
3	PCA	10	0.993789
0	Full gene expression	20531	0.981366

Recommendation: use UMAP or 2D PCA for exploratory visualization, but use the full standardized expression vector, or a higher-dimensional PCA representation, for downstream prediction. PCA is best when interpretability matters because loadings identify influential genes; UMAP is best when the lab wants a visually clean map of local tumor neighborhoods. I would not treat the 2D plots alone as proof of diagnostic performance, because the classifier accuracy table is the more direct test.

Problem 5: Clustering Palmer Penguins

This notebook implements every part of Problem 5 from HW3 using `penguins.csv`. The clustering code below implements Lloyd's k-means algorithm from scratch, and scikit-learn is only used for the silhouette score requested in part (f).

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import silhouette_score

sns.set_theme(style="whitegrid", context="notebook")
plt.rcParams["figure.figsize"] = (7, 5)
plt.rcParams["axes.spines.top"] = False
plt.rcParams["axes.spines.right"] = False

FEATURES = ["bill_length_mm", "bill_depth_mm", "flipper_length_mm", "body_mass_g"]
df = pd.read_csv("penguins.csv")
```

(a) Exploratory Data Analysis

(a)(i) Basics

```
In [2]: print("Shape:", df.shape)
display(df.describe())

print("Penguins per species:")
display(df["species"].value_counts().rename("count").to_frame())

print("Penguins per island:")
display(df["island"].value_counts().rename("count").to_frame())
```

Shape: (344, 8)

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	year
count	342.000000	342.000000	342.000000	342.000000	344.000000
mean	43.921930	17.151170	200.915205	4201.754386	2008.029070
std	5.459584	1.974793	14.061714	801.954536	0.818356
min	32.100000	13.100000	172.000000	2700.000000	2007.000000
25%	39.225000	15.600000	190.000000	3550.000000	2007.000000
50%	44.450000	17.300000	197.000000	4050.000000	2008.000000
75%	48.500000	18.700000	213.000000	4750.000000	2009.000000
max	59.600000	21.500000	231.000000	6300.000000	2009.000000

Penguins per species:

	count
species	
Adelie	152
Gentoo	124
Chinstrap	68

Penguins per island:

	count
island	
Biscoe	168
Dream	124
Torgersen	52

(a)(ii) Missing values

The assignment asks us to drop rows with any missing value in the four numerical morphological features.

```
In [3]: missing_by_column = df.isna().sum().rename("missing_values").to_frame()
display(missing_by_column)

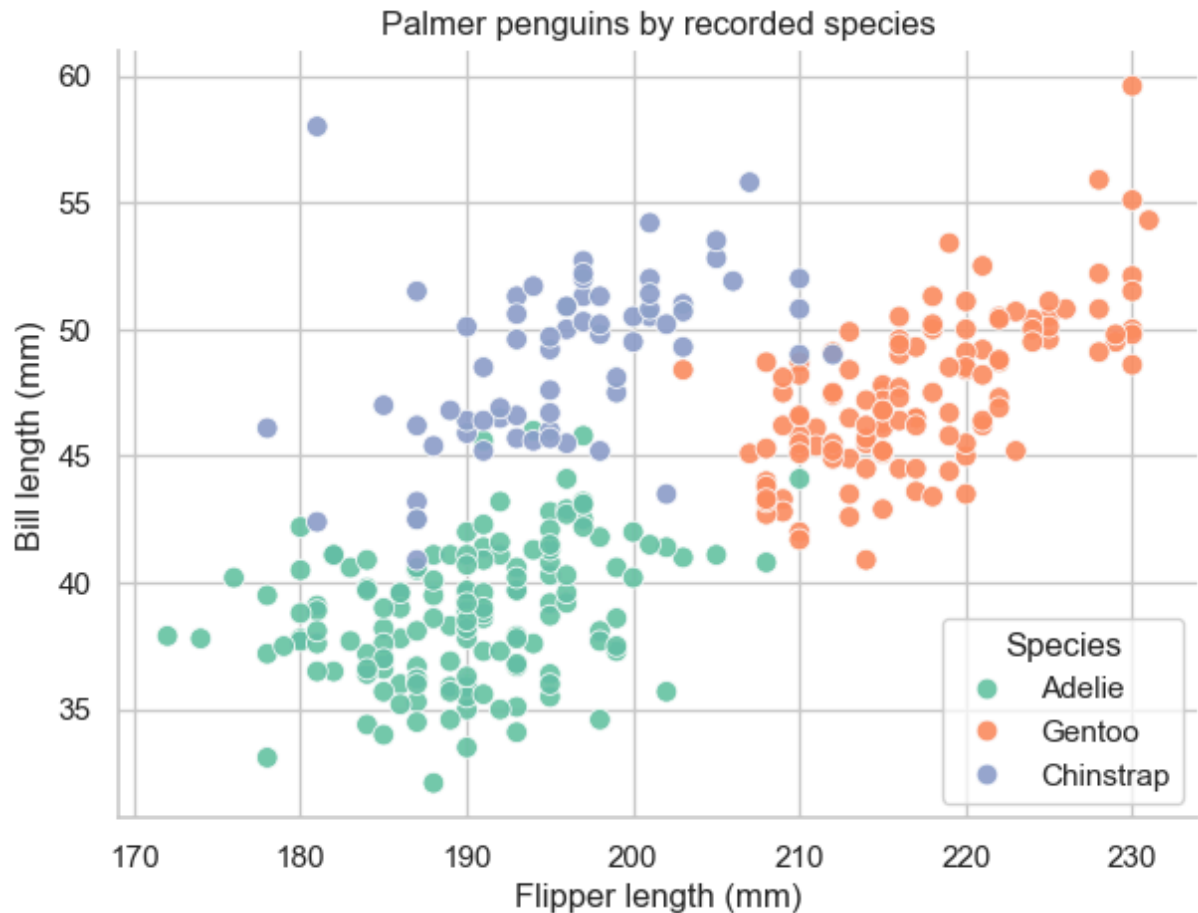
penguins = df.dropna(subset=FEATURES).copy().reset_index(drop=True)
print(f"Rows remaining after dropping NaN values in {FEATURES}: {len(penguins)}")
```

missing_values	
species	0
island	0
bill_length_mm	2
bill_depth_mm	2
flipper_length_mm	2
body_mass_g	2
sex	11
year	0

Rows remaining after dropping NaN values in ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g']: 342

(a)(iii) Visible clusters

```
In [4]: fig, ax = plt.subplots(figsize=(7, 5))
sns.scatterplot(
    data=penguins,
    x="flipper_length_mm",
    y="bill_length_mm",
    hue="species",
    palette="Set2",
    s=60,
    alpha=0.9,
    ax=ax,
)
ax.set_title("Palmer penguins by recorded species")
ax.set_xlabel("Flipper length (mm)")
ax.set_ylabel("Bill length (mm)")
ax.legend(title="Species")
plt.show()
```



There are visible groups, but not three perfectly separated blobs in this projection. Gentoo is the easiest to separate because it tends to have much longer flippers, while Adelie and Chinstrap overlap more in the flipper-length / bill-length view.

(b) Preprocessing

(b)(i) Pick the features

```
In [5]: X_raw = penguins[FEATURES].to_numpy(dtype=float)
species = penguins["species"].copy()

print("Feature matrix shape:", X_raw.shape)
display(penguins[FEATURES].head())
```

Feature matrix shape: (342, 4)

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
0	39.1	18.7	181.0	3750.0
1	39.5	17.4	186.0	3800.0
2	40.3	18.0	195.0	3250.0
3	36.7	19.3	193.0	3450.0
4	39.3	20.6	190.0	3650.0

(b)(ii) Why standardize

k-means uses Euclidean distance, so features measured on larger numerical scales dominate the objective. Here, body mass is measured in thousands of grams while bill measurements are in millimeters, so without standardization the clustering would mostly optimize body mass and underweight bill and flipper geometry.

(b)(iii) Standardize

```
In [6]: feature_means = X_raw.mean(axis=0)
feature_stds = X_raw.std(axis=0, ddof=0)
X = (X_raw - feature_means) / feature_stds

standardized_summary = pd.DataFrame(
    {
        "mean_after_standardization": X.mean(axis=0),
        "std_after_standardization": X.std(axis=0, ddof=0),
    },
    index=FEATURES,
)
display(standardized_summary)
```

	mean_after_standardization	std_after_standardization
bill_length_mm	1.662088e-16	1.0
bill_depth_mm	4.155221e-16	1.0
flipper_length_mm	-8.310441e-16	1.0
body_mass_g	8.310441e-17	1.0

(c) Implement k-means from scratch

```
In [7]: def kmeans(X, k, max_iter=100, seed=0):
# returns (centroids, labels, inertia)
X = np.asarray(X, dtype=float)
n_samples = X.shape[0]
if not 1 <= k <= n_samples:
```

```

        raise ValueError("k must be between 1 and the number of rows in X")

    rng = np.random.default_rng(seed)
    initial_indices = rng.choice(n_samples, size=k, replace=False)
    centroids = X[initial_indices].copy()

    labels = None
    inertia_history = []

    for _ in range(max_iter):
        squared_distances = ((X[:, None, :] - centroids[None, :, :]) ** 2).sum(axis=2)
        new_labels = squared_distances.argmin(axis=2)

        new_centroids = centroids.copy()
        for cluster_id in range(k):
            cluster_mask = new_labels == cluster_id
            if cluster_mask.any():
                new_centroids[cluster_id] = X[cluster_mask].mean(axis=0)

        inertia = float(((X - new_centroids[new_labels]) ** 2).sum())
        inertia_history.append(inertia)

        if labels is not None and np.array_equal(new_labels, labels):
            labels = new_labels
            centroids = new_centroids
            break

        labels = new_labels
        centroids = new_centroids

    kmeans.inertia_history_ = inertia_history
    return centroids, labels, inertia_history[-1]

def best_kmeans(X, k, seeds=range(10), max_iter=100):
    best = None
    for seed in seeds:
        centroids, labels, inertia = kmeans(X, k, max_iter=max_iter, seed=seed)
        result = {
            "seed": seed,
            "centroids": centroids,
            "labels": labels,
            "inertia": inertia,
            "history": kmeans.inertia_history_.copy(),
        }
        if best is None or inertia < best["inertia"]:
            best = result
    return best

```

(c)(ii) Sanity check on the objective

```

In [8]: _, _, example_inertia = kmeans(X, k=3, seed=0)
        history = np.array(kmeans.inertia_history_)

        print(f"Final inertia for seed 0: {example_inertia:.3f}")

```

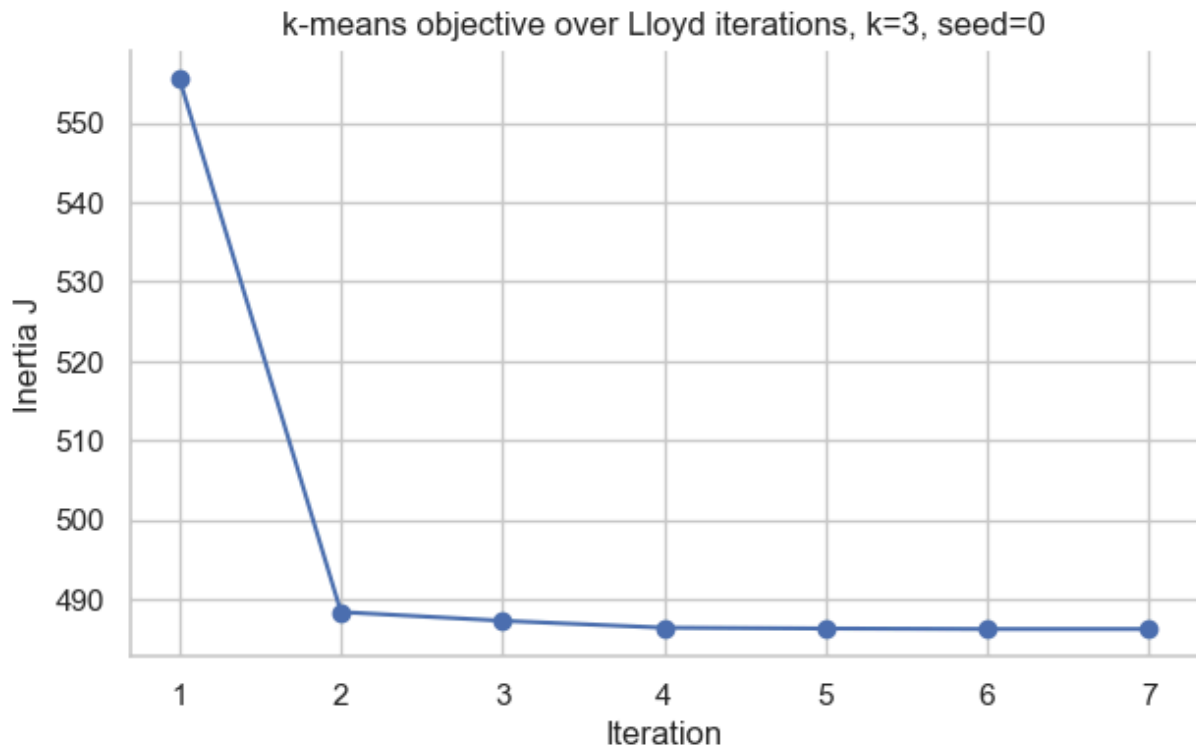
```
print("Inertia history:", np.round(history, 3))
print("Monotone non-increasing:", bool(np.all(np.diff(history) <= 1e-10)))

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(np.arange(1, len(history) + 1), history, marker="o")
ax.set_title("k-means objective over Lloyd iterations, k=3, seed=0")
ax.set_xlabel("Iteration")
ax.set_ylabel("Inertia J")
plt.show()
```

Final inertia for seed 0: 486.202

Inertia history: [555.582 488.346 487.246 486.369 486.268 486.202 486.202]

Monotone non-increasing: True



The objective is monotone non-increasing on this run, as expected for Lloyd's algorithm.

(d) Cluster and validate

Because k-means can get stuck in a poor local minimum, the main `k=3` clustering below uses the best inertia among seeds 0 through 9. Part (g) studies that initialization sensitivity directly.

```
In [9]: main_run = best_kmeans(X, k=3, seeds=range(10))
centroids = main_run["centroids"]
cluster_labels = main_run["labels"]
final_inertia = main_run["inertia"]

print(f"Best seed among 0..9: {main_run['seed']}")
print(f"Final inertia for k=3: {final_inertia:.3f}")
```


Best seed among 0..9: 2
 Final inertia for k=3: 379.393

(d)(ii) Did we rediscover the species?

```
In [10]: cluster_species_table = pd.crosstab(
    pd.Series(cluster_labels, name="cluster"),
    species.rename("species"),
)
purity = cluster_species_table.max(axis=1).sum() / cluster_species_table.to_numpy()

display(cluster_species_table)
print(f"Clustering purity: {purity:.3f}")
```

species **Adelie** **Chinstrap** **Gentoo**

cluster

0	24	63	0
1	127	5	0
2	0	0	123

Clustering purity: 0.915

The clustering strongly recovers Gentoo, and it partially separates Adelie from Chinstrap. The main confusion is between Adelie and Chinstrap, which matches the species-colored scatter plot from part (a).

(d)(iii) Cluster profiles in original units

```
In [11]: centroids_original_units = centroids * feature_stds + feature_means
centroid_table = pd.DataFrame(centroids_original_units, columns=FEATURES)
centroid_table.index.name = "cluster"
display(centroid_table.round(2))
```

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
cluster				
0	47.53	18.76	196.90	3902.01
1	38.21	18.11	188.40	3584.66
2	47.50	14.98	217.19	5076.02

Cluster 0 represents penguins with relatively long bills, deep bills, medium flippers, and medium body mass; this profile is closest to Chinstrap.

Cluster 1 represents smaller penguins with shorter bills, deep bills, shorter flippers, and lower body mass; this profile is closest to Adelie.

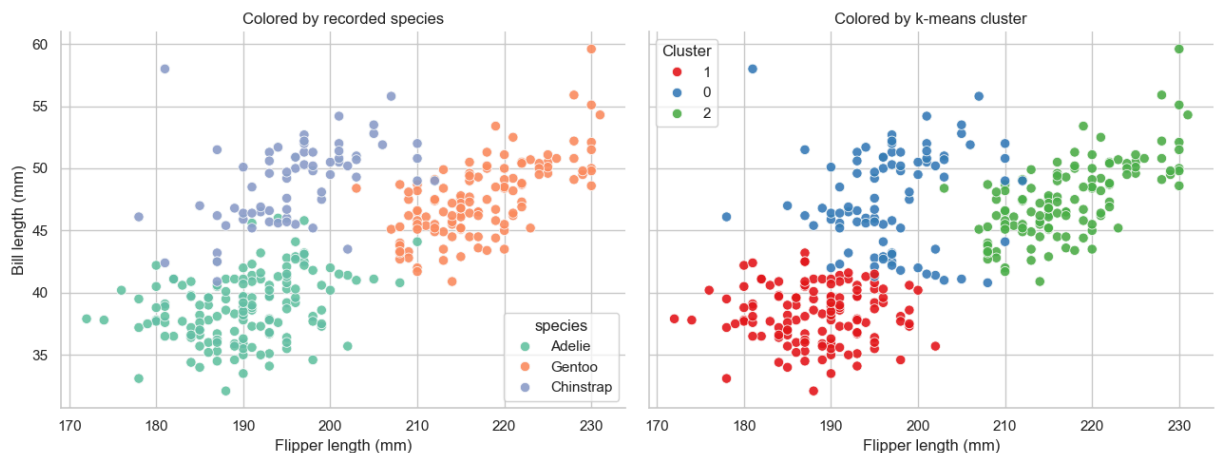
Cluster 2 represents larger penguins with long flippers, high body mass, and shallower bills; this profile is closest to Gentoo.

(d)(iv) Visualize clusters

```
In [12]: plot_df = penguins.copy()
plot_df["cluster"] = cluster_labels.astype(str)

fig, axes = plt.subplots(1, 2, figsize=(13, 5), sharex=True, sharey=True)
sns.scatterplot(
    data=plot_df,
    x="flipper_length_mm",
    y="bill_length_mm",
    hue="species",
    palette="Set2",
    s=55,
    alpha=0.9,
    ax=axes[0],
)
axes[0].set_title("Colored by recorded species")
axes[0].set_xlabel("Flipper length (mm)")
axes[0].set_ylabel("Bill length (mm)")

sns.scatterplot(
    data=plot_df,
    x="flipper_length_mm",
    y="bill_length_mm",
    hue="cluster",
    palette="Set1",
    s=55,
    alpha=0.9,
    ax=axes[1],
)
axes[1].set_title("Colored by k-means cluster")
axes[1].set_xlabel("Flipper length (mm)")
axes[1].set_ylabel("Bill length (mm)")
axes[1].legend(title="Cluster")
plt.tight_layout()
plt.show()
```



The cluster-colored plot looks similar to the species-colored plot for Gentoo, but differs in the Adelie/Chinstrap overlap region. k-means makes a geometric split there, while the species labels are not perfectly separated in this two-dimensional projection.

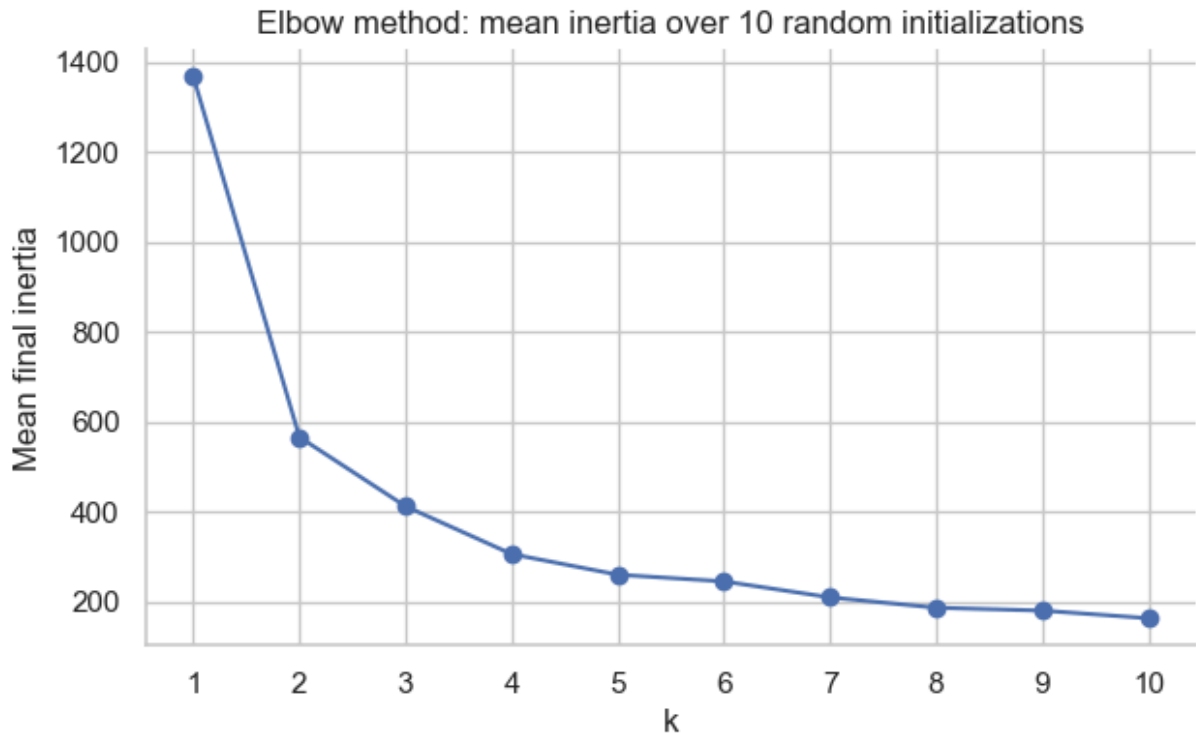
(e) The elbow method

```
In [13]: elbow_rows = []
for k in range(1, 11):
    inertias = [kmeans(X, k=k, seed=seed)[2] for seed in range(10)]
    elbow_rows.append(
        {
            "k": k,
            "mean_inertia": np.mean(inertias),
            "std_inertia": np.std(inertias, ddof=0),
            "min_inertia": np.min(inertias),
        }
    )

elbow_df = pd.DataFrame(elbow_rows)
display(elbow_df.round(3))

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(elbow_df["k"], elbow_df["mean_inertia"], marker="o")
ax.set_xticks(range(1, 11))
ax.set_title("Elbow method: mean inertia over 10 random initializations")
ax.set_xlabel("k")
ax.set_ylabel("Mean final inertia")
plt.show()
```

	k	mean_inertia	std_inertia	min_inertia
0	1	1368.000	0.000	1368.000
1	2	565.708	0.000	565.708
2	3	411.627	48.865	379.393
3	4	305.269	5.222	300.400
4	5	259.483	27.127	232.597
5	6	244.537	30.338	205.127
6	7	209.003	30.523	187.155
7	8	185.960	22.712	171.368
8	9	179.548	23.879	160.739
9	10	162.613	9.241	152.667



The clearest bend is at $k = 2$: moving from one cluster to two gives a very large drop, and later improvements are more gradual. There is still a smaller improvement at $k = 3$, which is biologically meaningful because the dataset has three recorded species, but the elbow alone says the strongest morphological split is closer to two groups.

(f) Silhouette score

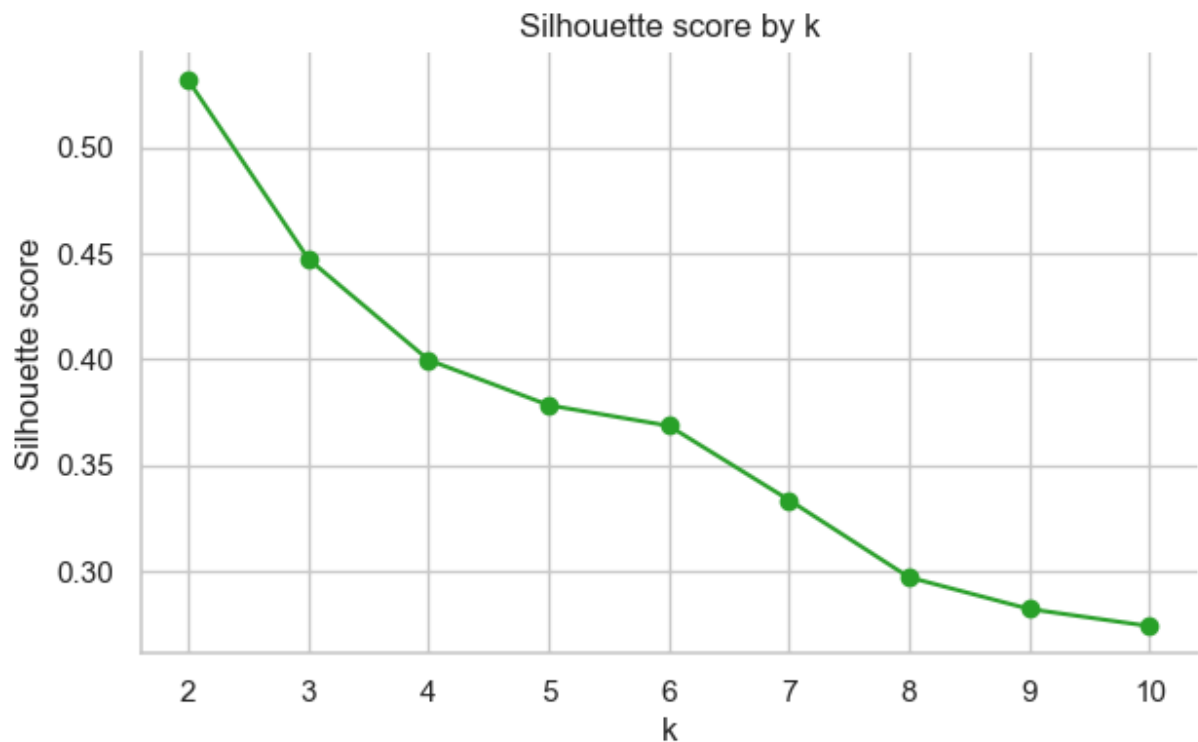
```
In [14]: silhouette_rows = []
for k in range(2, 11):
    run = best_kmeans(X, k=k, seeds=range(10))
    score = silhouette_score(X, run["labels"])
    silhouette_rows.append(
        {
            "k": k,
            "best_seed": run["seed"],
            "best_inertia": run["inertia"],
            "silhouette": score,
        }
    )

silhouette_df = pd.DataFrame(silhouette_rows)
display(silhouette_df.round(3))

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(silhouette_df["k"], silhouette_df["silhouette"], marker="o", color="tab:green")
ax.set_xticks(range(2, 11))
ax.set_title("Silhouette score by k")
ax.set_xlabel("k")
ax.set_ylabel("Silhouette score")
plt.show()
```

```
best_silhouette_row = silhouette_df.loc[silhouette_df["silhouette"].idxmax()]
print(
    f"Best silhouette occurs at k={int(best_silhouette_row['k'])} "
    f"with score {best_silhouette_row['silhouette']:.3f}."
)
```

	k	best_seed	best_inertia	silhouette
0	2	0	565.708	0.532
1	3	2	379.393	0.447
2	4	1	300.400	0.400
3	5	6	232.597	0.378
4	6	4	205.127	0.369
5	7	6	187.155	0.334
6	8	9	171.368	0.297
7	9	4	160.739	0.282
8	10	1	152.667	0.274



Best silhouette occurs at k=2 with score 0.532.

The silhouette score also prefers `k = 2`, so it agrees with the strongest elbow signal. I would trust that for the unsupervised geometry of these four standardized features: the strongest split is Gentoo versus non-Gentoo, while Adelie and Chinstrap are less cleanly separated by k-means.

(g) Sensitivity to initialization

```
In [15]: seed_rows = []
for seed in range(50):
    centroids_seed, labels_seed, inertia_seed = kmeans(X, k=3, seed=seed)
    seed_rows.append({"seed": seed, "inertia": inertia_seed, "labels": labels_seed})

seed_df = pd.DataFrame(seed_rows)
display(seed_df[["seed", "inertia"]].describe().round(3))

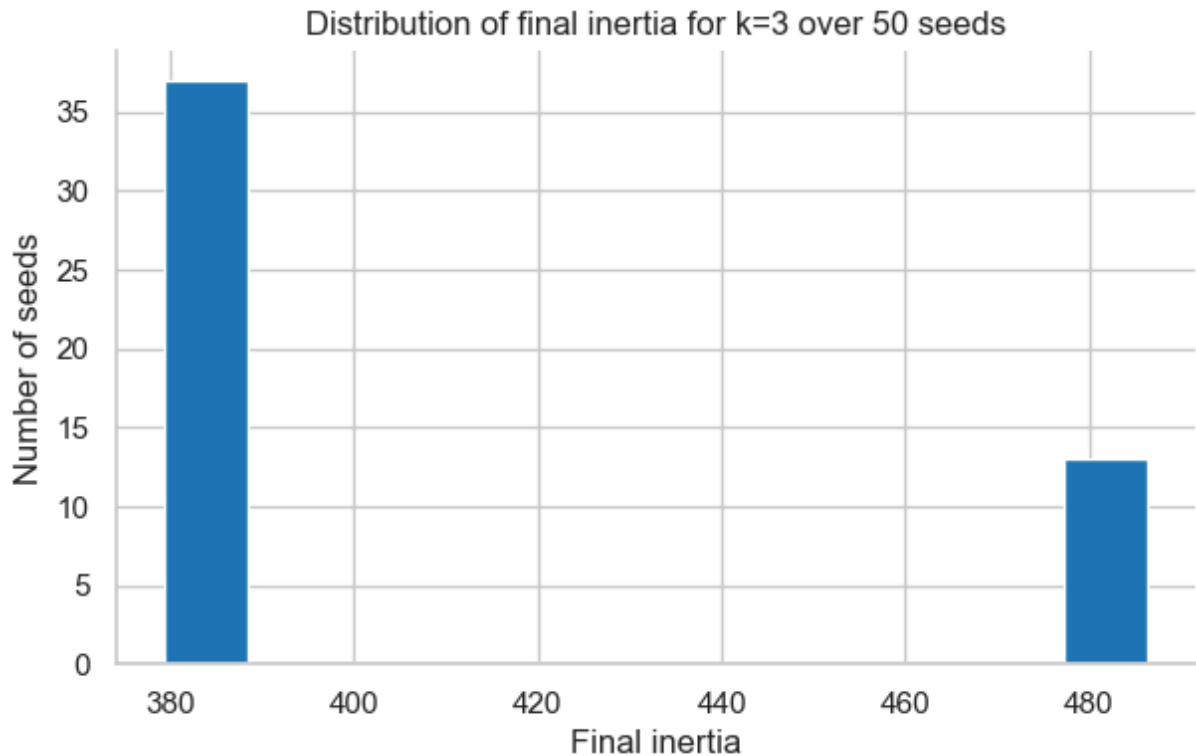
best_seed = int(seed_df.loc[seed_df["inertia"].idxmin(), "seed"])
worst_seed = int(seed_df.loc[seed_df["inertia"].idxmax(), "seed"])
best_seed_inertia = float(seed_df.loc[seed_df["inertia"].idxmin(), "inertia"])
worst_seed_inertia = float(seed_df.loc[seed_df["inertia"].idxmax(), "inertia"])

print(f"Lowest-inertia seed: {best_seed} with inertia {best_seed_inertia:.3f}")
print(f"Highest-inertia seed: {worst_seed} with inertia {worst_seed_inertia:.3f}")
print(f"Spread: {worst_seed_inertia - best_seed_inertia:.3f}")

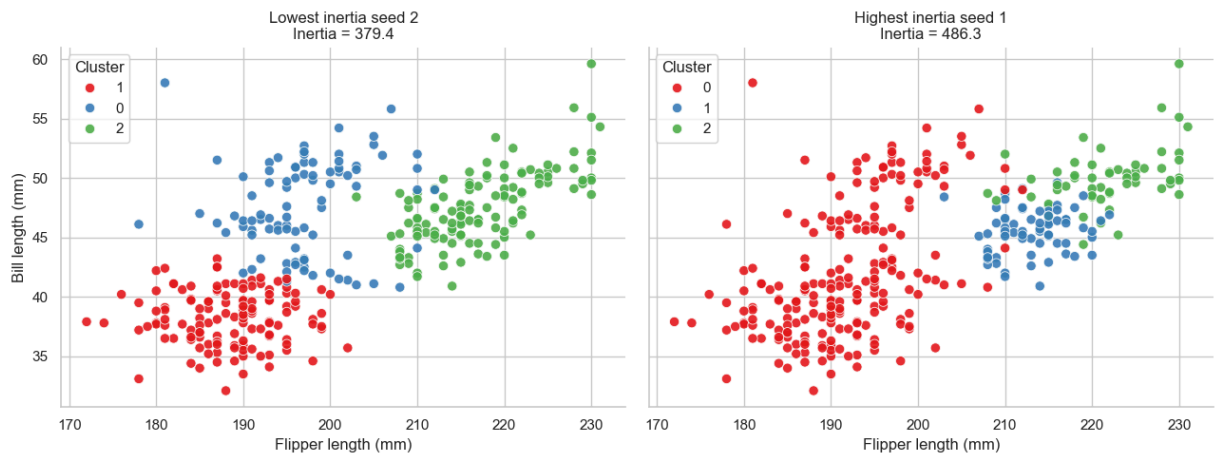
fig, ax = plt.subplots(figsize=(7, 4))
ax.hist(seed_df["inertia"], bins=12, color="tab:blue", edgecolor="white")
ax.set_title("Distribution of final inertia for k=3 over 50 seeds")
ax.set_xlabel("Final inertia")
ax.set_ylabel("Number of seeds")
plt.show()
```

	seed	inertia
count	50.000	50.000
mean	24.500	407.466
std	14.577	47.193
min	0.000	379.393
25%	12.250	379.393
50%	24.500	379.403
75%	36.750	459.985
max	49.000	486.297

Lowest-inertia seed: 2 with inertia 379.393
 Highest-inertia seed: 1 with inertia 486.297
 Spread: 106.905



```
In [16]: fig, axes = plt.subplots(1, 2, figsize=(13, 5), sharex=True, sharey=True)
for ax, seed, title in [
    (axes[0], best_seed, f"Lowest inertia seed {best_seed}"),
    (axes[1], worst_seed, f"Highest inertia seed {worst_seed}"),
]:
    _, labels_seed, inertia_seed = kmeans(X, k=3, seed=seed)
    temp = penguins.copy()
    temp["cluster"] = labels_seed.astype(str)
    sns.scatterplot(
        data=temp,
        x="flipper_length_mm",
        y="bill_length_mm",
        hue="cluster",
        palette="Set1",
        s=55,
        alpha=0.9,
        ax=ax,
    )
    ax.set_title(f"{title}\nInertia = {inertia_seed:.1f}")
    ax.set_xlabel("Flipper length (mm)")
    ax.set_ylabel("Bill length (mm)")
    ax.legend(title="Cluster")
plt.tight_layout()
plt.show()
```



The spread is substantial: the worst seed among the first 50 has much higher inertia than the best seed. The two visualizations disagree more than just a few boundary points; the poor run splits Gentoo into two clusters and merges most Adelie and Chinstrap penguins, while the low-inertia run separates Gentoo and gives a more meaningful Adelie/Chinstrap split.

(h) Summary and recommendation

Using only the four standardized morphological measurements, the strongest unsupervised structure appears to be two groups, not three: both the elbow curve and the silhouette score point most clearly to $k = 2$. A three-cluster solution is still biologically useful because it recovers the recorded species labels with about 91.5% purity when using the best-inertia run among seeds 0 through 9, with Gentoo almost perfectly separated and most remaining confusion between Adelie and Chinstrap. I would report that the data strongly separates Gentoo from the other penguins, while the Adelie/Chinstrap boundary is weaker and more sensitive to initialization. Because Lloyd's algorithm can land in different local minima, any reported k-means result should use multiple random initializations and keep the lowest-inertia run. A limitation is that this analysis uses only four morphological measurements after dropping rows with missing numerical values, so it may miss ecological or genetic information that could distinguish species more reliably.